MossOS

设计文档

参赛院校：重庆邮电大学

参赛队名：每天睡满八小时

队伍成员：吴勋、张可欣

2026 年 6 月

目录

1 概述 1
1.1 项目简介 1
1.2 系统架构 1
1.3 支持平台 2
1.4 从初赛到决赛 2
1.5 代码结构 3
2 进程与线程管理 4
2.1 概述 4
2.2 核心设计思想 4
2.3 关键数据结构 5
2.4 进程与线程创建 6
2.5 程序执行 8
2.6 退出与资源回收 9
2.7 进程组与会话 9
2.8 权限与进程控制 11
2.9 本章小结 12
3 调度与同步机制 13
3.1 概述 13
3.2 任务调度机制 14
3.3 阻塞与唤醒机制 17
3.4 通用阻塞机制的应用 19
3.5 Futex 同步机制 20
3.6 本章小结 22
4 内存管理 23
4.1 概述 23
4.2 栈自动增长 24
4.3 mremap 迁移 24
4.4 MADV_WIPEONFORK 25
4.5 /proc/[pid]/maps 25
4.6 PROT_WRITE 与 SIGBUS 25
4.7 本章小结 25
5 文件系统 26
5.1 概述 26
5.2 挂载子系统 27
5.3 inode 元数据缓存 29
5.4 Folio 文件页缓存 29
5.5 扩展属性 30
5.6 文件事件通知 30
5.7 文件 I/O 测试 32
5.8 本章小结 34
6 信号机制 35
6.1 StarryOS 的信号机制 35
6.2 进程可暂停状态 35
6.3 信号驱动的 wait 语义 37
6.4 父进程如何感知状态变化 38
6.5 kill 的信号权限校验 39

6.6 小结 39
7 网络模块 41
7.1 StarryOS 的网络栈 41
7.2 地址族支持与兼容性补齐 41
7.3 Buffer 扩展 42
7.4 Loopback 路径优化 43
7.5 初赛阶段性能记录 44
7.6 小结 46
8 多架构硬件平台适配与设备驱动 48
8.1 平台适配概述 48
8.2 设备树与硬件资源发现 49
8.3 中断与多核事件机制 50
8.4 设备驱动 51
9 总结与展望 53
9.1 工作总结 53
9.2 未来工作 53

第 1 章 概述

1.1 项目简介

MossOS 是用 Rust 语言实现的 Linux ABI 兼容宏内核，基于 ArceOS 组件化框架，从 StarryOS 演化而来。系统复用 ArceOS 的任务、内存、文件系统、网络和硬件抽象组件，并在其上实现进程线程、用户地址空间、信号、文件描述符和 Linux 系统调用语义。

MossOS 支持 RISC-V64 和 LoongArch64 两种指令集，内核主体在四个平台目标间共享：RISC-V QEMU virt、LoongArch QEMU virt、StarFive VisionFive 2 开发板 和 Loongson LS2K1000LA 开发板。平台差异由统一接口、定制平台 crate 和设备驱动共同承担。

1.2 系统架构

MossOS 的整体结构如 图 1-1 所示。用户程序通过 Linux ABI 系统调用进入内核；内核按功能划分为任务与执行、地址空间、文件存储、I/O 与事件、IPC 与同步、网络、资源管理和运行支撑等功能域；下层由 ArceOS 组件和具体平台实现提供调度、内存、驱动与硬件能力。图中各功能域表示系统的职责划分，不代表其已覆盖 Linux 的全部接口。

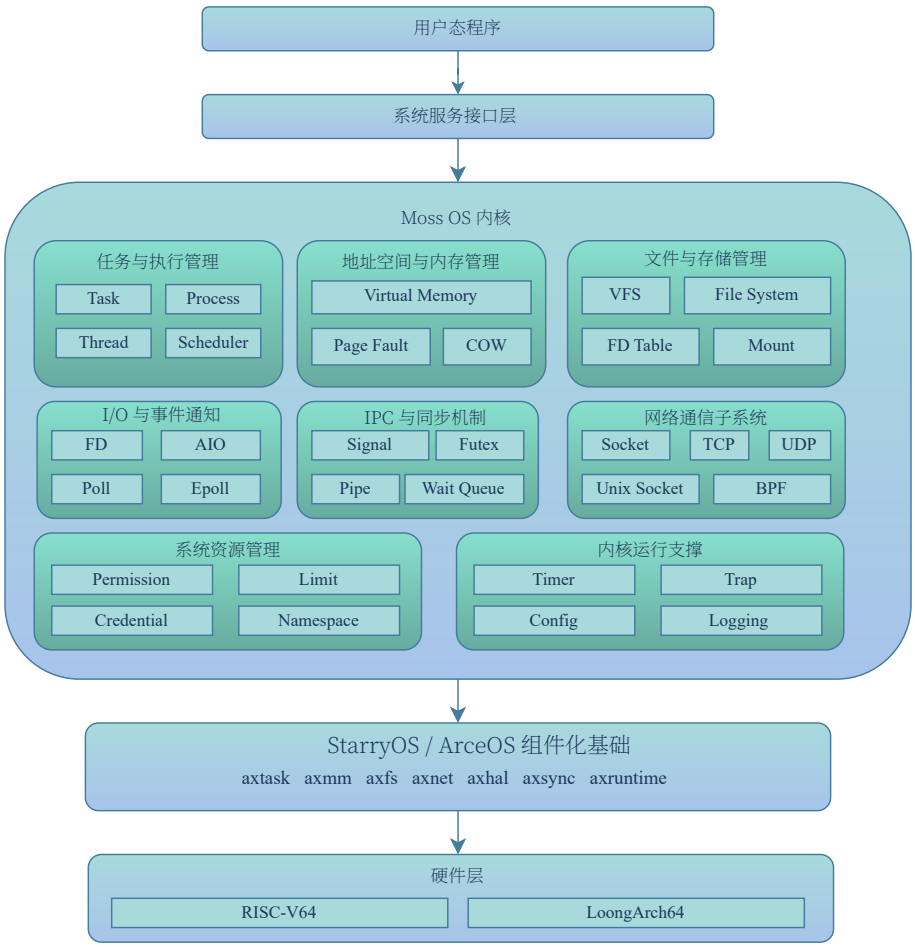


图 1-1　MossOS 系统整体架构

`axcpu` 封装不同指令集的寄存器、陷阱和上下文切换操作；`axplat` 定义中断、时钟、串口、内存和启动等平台接口；四个平台 crate 分别实现这些接口。`axhal` 向内核提供统一的硬件访问入口，`axruntime` 负责主从核、内存、调度器、中断和驱动的初始化顺序。决赛期间进一步从 DTB 读取 CPU 与内存拓扑，使得同一套多核运行时能够适配 QEMU 和物理开发板。

1.3 支持平台

平台	决赛验证配置	主要硬件路径	验证状态
RISC-V QEMU virt	8 核、8 GiB	SBI、PLIC、VirtIO MMIO	cagent 与 BuildStorm 完整运行
LoongArch QEMU virt	12 核、8 GiB	EIOINTC、IOCSR IPI、VirtIO PCI	cagent 与 BuildStorm 完整运行
VisionFive 2	4 核、4 GiB	PLIC、UART、SD/MMC、GMAC1	已完成实板启动、存储、网络与用户程序验证
LS2K1000LA	2 核目标	LIOINTC、AHCI、GMAC、RTC	已通过交叉构建和状态机测试，尚未实板验证

表 1-1　MossOS 决赛版平台支持与验证状态

1.4 从初赛到决赛

1.4.1 初赛成绩

截至 2026 年 6 月 30 日 20:22，MossOS（队名“每天睡满八小时”）在初赛排行榜上位列第 3，综合得分 2601.18。当时进程、内存、文件系统、信号、网络和设备驱动已具备主干功能，并可在 QEMU 上运行 BusyBox shell、Lua 和 libc 测试程序。

#	用户名	队伍	学校	提交次数 (ASC)	最后提交时间 (ASC)	basic-glibc-la	basic-glibc-rv	basic-musl-la	basic-musl-rv	busybox-glibc-la	busybox-glibc-rv	busybox-musl-la	bu mu
1	T202610486999845	alREDy	武汉大学	262	2026-06-28 16:24:33	100.0	102.0	102.0	102.0	54.0	54.0	54.0	54.
2	T202610459999616	PulseOS	郑州大学	164	2026-06-29 11:25:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.
3	T202610617999569	每天睡满八小时	重庆邮电大学	200	2026-06-24 02:43:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54

图 1-2　初赛排行榜（截至 2026 年 6 月 30 日）

1.4.2 决赛成绩

截至 2026 年 8 月 12 日，队伍在决赛线上排行榜中位列第 2，总分 758.2。RISC-V64 与 LoongArch64 的 BuildStorm 得分均为 180.0，cagent-glibc 得分均为 199.1。

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T2026104879910415	维塔立体	华中科技大学	46	180.0	180.0	199.1	199.1	758.2000
2	T202610617999569	每天睡满八小时	重庆邮电大学	60	180.0	180.0	199.1	199.1	758.2000
3	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
4	T2026104869910383	洗剪吹	武汉大学	77	180.0	169.3	199.1	199.1	747.5000
5	T202610459999616	PulseOS	郑州大学	19	180.0	151.3	199.1	199.1	729.5000

图 1-3　决赛线上排行榜（截至 2026 年 8 月 12 日）

1.4.3 决赛阶段主要进展

方向	主要进展
多核正确性	从 DTB 获取 CPU 与内存拓扑，完善从核启动、远端唤醒、跨核 TLB 刷新和多核调度
进程与同步	完善 `vfork`、线程组 leader 发起的 `execve`、`rseq`、Futex、管道、信号与任务退出的并发语义
物理平台	完成 VisionFive 2 四核实板路径，并建立 LS2K1000LA 的 FDT 双核平台与基础外设驱动
高并发构建	完成 BuildStorm 专项性能优化，设计与评测数据见专项报告

表 1-2　MossOS 决赛阶段的主要进展

BuildStorm 的性能分析、优化设计和评测数据已在《MossOS 内核设计优化实现文档——BuildStorm》中独立说明，本报告不再重复展开。

1.5 代码结构

```
MossOS/
├── src/                                  # 内核程序入口
├── kernel/src/
│   ├── task/                            # 进程、线程与信号
│   ├── mm/                              # 地址空间、缺页与 TLB
│   ├── file/                            # 文件抽象与描述符表
│   ├── pseudofs/                        # procfs、sysfs 与 devfs
│   ├── syscall/                         # Linux 系统调用
│   └── ipi.rs                           # 跨核回调
├── vendor/
│   ├── axcpu/ axhal/ axruntime/         # CPU、硬件抽象与运行时
│   ├── axtask/ axmm/ axfs/ axnet/       # 内核基础组件
│   ├── axdriver*/                       # 块设备与网络驱动
│   ├── axplat-riscv64-qemu-virt/        # RISC-V QEMU 平台
│   ├── axplat-loongarch64-qemu-virt/    # LoongArch QEMU 平台
│   ├── axplat-riscv64-visionfive2/      # VisionFive 2 平台
│   └── axplat-loongarch64-ls2k1000/     # LS2K1000LA 平台
├── third_party/                           # 第三方库
├── docs/                                  # 设计、评测与上板文档
└── Makefile                               # 统一构建入口
```

`vendor/` 保存了针对 MossOS 定制的 ArceOS 组件。本报告后续章节依次介绍进程线程、调度同步、内存、文件系统、信号、网络以及平台与设备驱动设计。

第 2 章 进程与线程管理

进程与线程管理负责组织用户程序的执行主体及其资源，向内连接地址空间、文件描述符和信号等内核对象，向外提供创建、执行、退出、等待和控制接口。

2.1 概述

StarryOS 已经建立了可运行的进程线程框架。每个用户执行流对应一个 `Thread`，同一进程中的线程共同持有 `ProcessData`。前者保存线程私有状态，后者保存地址空间、文件资源、进程级信号和父子关系等共享状态。在功能上，StarryOS 已经提供了 `clone`、`clone3`、`execve`、`exit` 和 `waitpid`，x86_64 配置下还提供独立的 `fork` 入口。这些接口能够完成进程创建、程序映像替换、线程退出和僵尸进程回收，进程组、会话和 pidfd 也已有基本实现。

然而，基线只处理了“创建子进程、运行、退出、父进程回收”这条主路径，接口组合复杂后问题便逐渐暴露：clone 的标志约束和失败回滚不够严密，exec 缺少 `execveat` 及执行权限检查，wait 无法完整表达停止和继续状态，UID/GID 部分接口直接返回成功而没有保存身份变化，ptrace 和 pidfd 的观测链路也尚未打通。换句话说，基线能建立进程，却难以把进程的身份、状态和生命周期完全串联起来。

MossOS 保留了 StarryOS 的主体结构，重点补齐这些接口之间的联系。主要增量如表所示：

内容	StarryOS 基线	MossOS 的增量
对象与状态	`Thread` 与 `ProcessData` 分离	补充凭据、停止/继续状态、子进程时间和 namespace 元数据
创建与执行	`clone3`、`execve` 已实现	补充 clone 参数校验与失败回滚，增加 `execveat` 与执行约束
退出与等待	线程退出、僵尸进程和 `waitpid`	完善资源清理，增加 `waitid`、状态事件和 pidfd 等待
管理接口	进程组、会话、基础凭据与 pidfd	修正全局登记，实现有状态凭据和最小 ptrace 控制链路

表 2-1　进程与线程模块的主要增量

2.2 核心设计思想

StarryOS 已经搭起了进程线程管理的基本框架，MossOS 的工作是在这套框架中进一步理清状态归属和对象关系：哪些数据属于单个线程，哪些资源应由整个进程共享；进程之间如何组织；身份信息怎样参与权限判断；一个任务又该如何从创建走到退出和回收。下面从这几个问题分别说明。

2.2.1 双层对象模型

进程和线程并不是两套互不相干的对象。MossOS 延续 StarryOS 的做法，将状态分为两层：`Thread` 表示单个执行流，`ProcessData` 表示线程共同拥有的进程资源。创建普通线程时复用 `ProcessData`；创建新进程时再根据 clone 标志复制或共享其中的资源。

这种划分回答了一个最基本的问题：某项状态究竟属于线程，还是属于整个进程。线程信号状态、`clear_child_tid`、robust list 和退出字段放在 `Thread` 中；地址空间、程序路径、文件表作用域、凭据和子进程集合放在 `ProcessData` 中。只有边界清楚，clone、exec 和 exit 才不会各自维护一份互相冲突的状态。

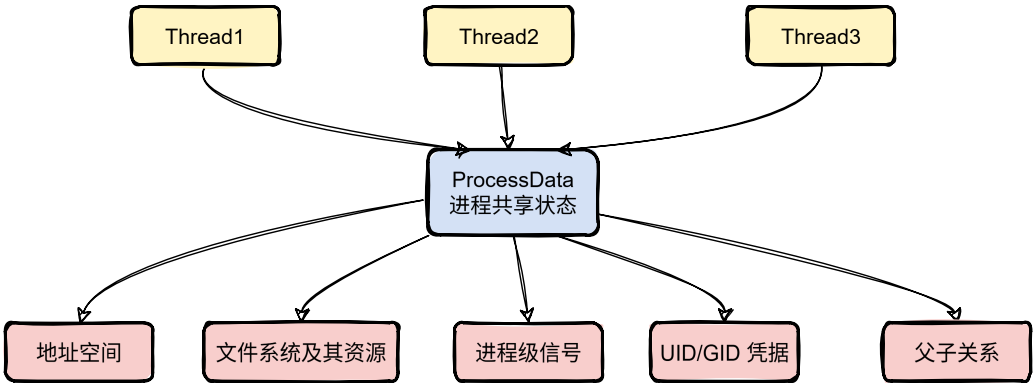


图 2-1　Thread 与 ProcessData 的双层对象模型

2.2.2　三类组织关系

MossOS 同时维护三类关系。

- 归属关系：多个线程通过共享 `ProcessData` 归属于同一进程。
- 亲缘关系：进程通过 parent 和 children 形成父子树，供退出托管和 wait 使用。
- 分组关系：进程依次归入进程组和会话，形成 PID、PGID、SID 三层标识。

这三类关系用途不同。线程组解决“哪些执行流共享资源”，父子树解决“谁负责回收谁”，进程组和会话解决“哪些进程作为一个作业被统一管理”。MossOS 没有把它们压进同一张表，而是分别维护，再通过全局索引提供查询。

2.2.3　三元凭据

MossOS 的进程身份设计参考了 Linux 的凭据模型：UID 不只保存一个值，而是拆成 real、effective、saved 三个字段，GID 采用相同结构。real ID 表示身份来源，effective ID 用于当前权限判断，saved ID 允许进程在受限范围内恢复先前身份。MossOS 将 UID/GID 三元组放入 `ProcessData`，让凭据真正参与程序执行、信号发送和 ptrace 权限检查。具体实现见 小节 2.8.1。

2.2.4　生命周期闭环

进程管理不是若干独立系统调用的集合，而是一条连续的生命周期：

创建 `clone` → 执行 `exec` → 退出 `exit` → 观察与回收 `wait`

创建阶段建立身份、资源和父子关系；exec 替换程序映像，但保留 PID、父子关系和进程组；exit 释放运行资源并留下退出状态；wait 读取状态并完成最终回收。MossOS 的多项修改都围绕这条闭环展开，尤其重视失败路径和中间状态，而不仅是系统调用成功时的主路径。

2.3　关键数据结构

2.3.1　Thread 与 ProcessData

`Thread` 和 `ProcessData` 是这部分最重要的两类数据，下面是关键代码片段。

```
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    clear_child_tid: AtomicUsize,
    robust_list_head: AtomicUsize,
    pub signal: Arc<ThreadSignalManager>,
```

```
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    aspace: RwLock<Arc<AddrSpaceLock>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // UID/GID 三元组
    uid: AtomicU32, euid: AtomicU32, suid: AtomicU32,
    gid: AtomicU32, egid: AtomicU32, sgid: AtomicU32,
}
```

代码 2-1　线程私有状态与进程共享状态

同一进程中的线程持有同一个 `Arc<ProcessData>`，因此能看到相同的地址空间、文件资源和凭据。`Thread` 自身则保存只与当前执行流相关的数据。MossOS 新增字段时继续遵循这一边界，没有另建一套进程控制块。

2.3.2　四张全局索引表

内核需要按 TID、PID、PGID 和 SID 查找对象。MossOS 使用四张弱引用表分别索引线程、进程、进程组和会话：

索引表	键	作用
`TASK_TABLE`	TID	查找具体线程和线程私有状态
`PROCESS_TABLE`	PID	查找进程共享资源和父子关系
`PROCESS_GROUP_TABLE`	PGID	查找进程组并执行组级操作
`SESSION_TABLE`	SID	查找会话及其进程组层级

表 2-2　进程线程对象的全局索引

表中保存弱引用，不会因为全局索引而阻止对象释放。MossOS 还修正了新建进程组和会话后的登记过程，避免对象已经创建，却无法通过 PGID 或 SID 查到。

2.4　进程与线程创建

2.4.1　fork 与统一的 clone 路径

MossOS 将任务创建统一到 clone 路径。在 x86_64 平台上，独立的 `sys_fork` 入口会调用 `sys_clone`，不设置资源共享标志，只指定子进程退出时发送 `SIGCHLD`；RISC-V 和 LoongArch 目标上的用户态则主要通过 `clone` 或 `clone3` 表达相同的创建语义。`clone` 和 `clone3` 还允许用户通过标志决定创建进程还是线程，以及资源如何继承。

这些入口最终都把参数整理为 `CloneArgs`，交给 `CloneArgs::do_clone` 执行。带有 `CLONE_THREAD` 时，新任务复用当前 `ProcessData`；否则调用 `Process::fork` 建立子进程及父子关系，再创建新的 `ProcessData`。地址空间、文件表、文件系统上下文和信号处理动作分别由 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS` 和 `CLONE_SIGHAND` 决定共享或复制。

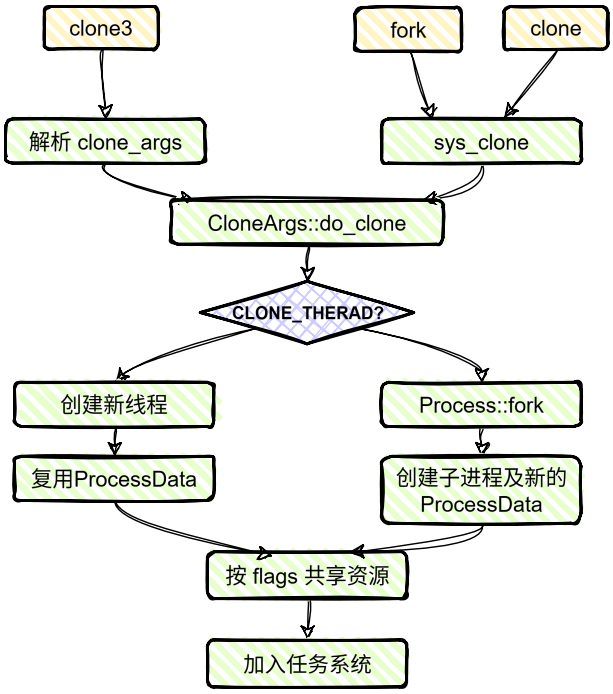


图 2-2　fork、clone 与 clone3 的统一创建路径

标志	作用
CLONE_THREAD	加入调用者的线程组，共享同一个 ProcessData
CLONE_VM	共享虚拟地址空间
CLONE_FILES	共享文件描述符表
CLONE_FS	共享文件系统上下文
CLONE_SIGHAND	共享信号处理动作表

表 2-3　进程线程创建中的主要 clone 标志

这些标志可以组合使用，但并非任意组合都合法。CLONE_THREAD 决定新任务是否仍属于原进程，其余标志主要控制资源如何继承，或指定创建前后的附加操作。

2.4.2　参数约束

MossOS 补充了 clone 标志之间的依赖关系，让不合法的参数组合直接报错，而不是创建一个运行后才暴露问题的异常线程/进程，把 bug 扼杀在摇篮里。例如，线程必须共享地址空间和信号处理动作，共享信号处理动作又必须共享地址空间：

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

代码 2-2　clone 标志组合校验

对于 clone3，MossOS 进一步检查结构体最小长度、栈地址与栈大小的配对关系，以及超出当前结构体版本的尾部字节。额外字节必须可读且全部为零；set_tid 和 cgroup 等扩展字段尚未生效。

2.4.3 创建失败回滚

如果子进程尚未开始运行，clone 就因后续操作失败，内核必须撤销已建立的父子关系和线程组记录，使系统调用返回错误时不留下半成品对象。

do_clone 会先建立进程对象、父子关系和线程组记录。若调用者请求 CLONE_PIDFD，内核还要分配文件描述符，并将编号写回用户地址。这些可失败操作完成后，新任务才提交给调度器并加入全局索引。

pidfd 分配可能因文件描述符表已满而失败，写回 pidfd 编号时也可能遇到无效用户指针。此时新任务尚未运行，也未加入 TASK_TABLE 和 PROCESS_TABLE，因此不能依赖正常的 exit 路径进行回收。

MossOS 因此在任务提交前统一处理失败路径：写回失败时关闭已分配的 pidfd，并撤销线程组记录和父子关系。因为全局索引在最后才登记，失败路径无需再从索引中删除半成品任务。

2.5 程序执行

2.5.1 execve 基线

StarryOS 的 execve 已经能够装载 ELF 文件，替换地址空间和用户上下文，更新程序路径与命令行，重置信号处理动作，并关闭带有 CLOEXEC 标志的文件描述符。exec 不创建新进程，因此 PID、父子关系、进程组和会话保持不变。基线的不足在于只能按路径执行文件，对执行权限、MS_NOEXEC 挂载点和正在写入的可执行文件缺少检查，也没有 execveat。

2.5.2 execveat 与执行检查

传统 execve 只能接收路径，绝对路径依赖进程看到的文件系统，相对路径又依赖当前工作目录。程序若已经打开了可信目录或目标文件，仍需重新拼接并解析路径，期间目录项可能被改名或替换。execveat 可以从稳定的目录文件描述符开始解析，或使用 AT_EMPTY_PATH 执行已经打开的文件，减少重复路径解析带来的竞态，也便于容器运行时、动态加载器等程序在受控目录中启动目标。

MossOS 增加该入口后，将 execve 与 execveat 汇入同一条 execve_common 路径，后续装载和状态更新仍由一套逻辑完成。在替换旧地址空间之前，MossOS 先完成执行条件检查：

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

代码 2-3　程序执行前的约束检查

这些检查全部通过后，内核才装载新程序并提交地址空间。若解析、权限或装载失败，原程序仍可继续运行。成功后，内核还会清除 `clear_child_tid`、robust list 和 `rseq` 等属于旧地址空间的线程状态。

2.6　退出与资源回收

2.6.1　单个线程退出与进程结束

一个进程可以包含多个线程，因此某个线程退出并不等于整个进程结束。内核先执行线程级退出清理，再把它从进程的线程集合中移除。清理内容包括 robust futex 和 `clear_child_tid`，其唤醒语义放在第三章的 futex 退出路径中说明。如果还有其他线程存活，共享的地址空间、文件表和 `ProcessData` 都继续保留。

只有最后一个线程退出时，内核才结束整个进程。此时进程进入僵尸状态，保存退出码，向父进程发送退出信号并唤醒正在等待的父进程。MossOS 还在这一阶段释放文件锁、租约、共享内存记录和 ptrace 关系。进程对象不会立即删除，要等父进程调用 `wait` 读取退出状态后才完成回收。

2.6.2　exit_group：结束全部线程

普通 `exit` 只结束调用线程，`exit_group` 则需要主动让进程中的全部线程退出。MossOS 先标记线程组正在整体退出，再向其他存活线程发送终止信号。各线程仍按上一节的路径完成清理，最后离开的线程负责执行进程级退出工作。这一区别对多线程程序尤其重要。

2.6.3　退出后的孤儿进程托管

进程结束后，还要处理它尚未退出的子进程。子进程不能随父进程一起删除——它们之后退出时需要有进程负责读取退出状态，否则将永远停留在僵尸状态占用内核资源。MossOS 沿用 StarryOS 的做法，把这些子进程从原父进程的 `children` 表中移出，改由 init 进程托管。如 图 2-3 所示，整个过程只修改父进程指针，子进程本身的 PID、PGID 和 SID 均保持不变，仍可继续运行。它们以后成为僵尸进程时，由 init 调用 `wait` 完成回收。

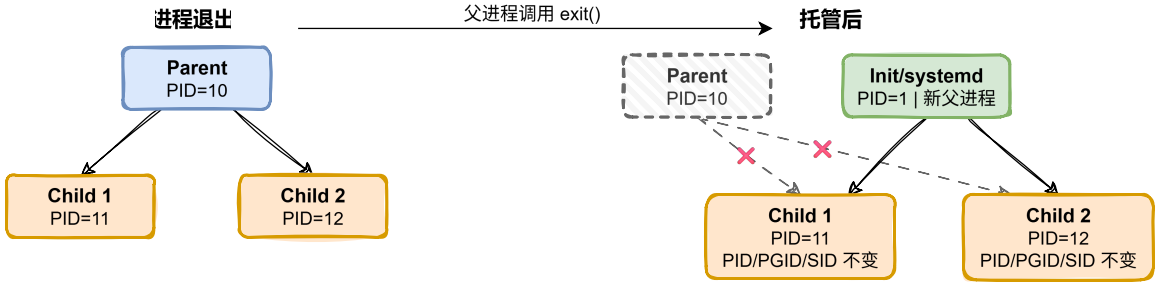


图 2-3　孤儿进程托管示意图

2.7　进程组与会话

2.7.1　PID、PGID 与 SID

进程组和会话用于组织多个相关进程。每个进程有 PID，所属进程组有 PGID，进程组所属会话有 SID。通常，组长的 PID 等于 PGID，会话首进程的 PID 等于 SID。这种层级常见于 Shell：一条管道中的多个进程属于同一进程组，一个终端登录环境中的多个进程组属于同一会话。它们仍是独立进程，但可以按组查询、等待或发送信号。

在实现上，MossOS 沿用 StarryOS 的对象关系：`Process` 保存自身 PID 和所属的 `ProcessGroup`，`ProcessGroup` 保存 PGID、所属 `Session` 及组内进程，`Session` 再保存 SID 和会话内的进程组。

```
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

代码 2-4　进程、进程组与会话的对象关系

查询一个进程的三个标识时，内核沿对象引用逐层取得结果：先按 PID 找到进程对象，再沿所属关系取得进程组和会话标识。反向操作则依靠成员表完成——向进程组发送信号时，内核遍历 `ProcessGroup::processes`，逐个找到组内进程。

2.7.2　进程组创建与迁移

StarryOS 已提供 `setpgid`、`getpgid` 等基础接口，但新建的进程组没有同步写入全局 PGID 索引，导致其他系统调用可能无法通过 PGID 找到该进程组。

MossOS 根据目标 PGID 分两种情况处理。目标 PGID 等于进程自身 PID 时，内核创建新进程组并立即登记到全局索引，此后组信号和按进程组等待等操作都能查到该对象。目标 PGID 指向已有进程组时，内核先从全局索引中找到目标组，检查它与当前进程是否属于同一会话，通过后将进程从旧组迁移到新组，同时更新全局索引。若目标组不存在或属于其他会话，则拒绝迁移。这样修改后，进程所属组、成员表和全局索引始终保持一致。

2.7.3　会话创建

`setsid` 会同时创建新会话和新进程组。MossOS 在创建成功后分别登记 `Session` 和 `ProcessGroup`：

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
```

```
    Ok(session.sid() as _)
}
```

代码 2-5　新会话与新进程组的创建和登记

2.8　权限与进程控制

2.8.1　UID/GID 三元组

进程的身份不能只用一个 UID 表示。系统既要知道它原本由谁启动，也要知道它此刻按谁的权限访问文件、执行程序或向其他进程发送信号。为此，MossOS 沿用 Linux 的三元凭据：real UID 记录原始用户，effective UID 用于当前权限检查，saved UID 保存此前可恢复的有效身份；GID 采用同样的结构。

这套设计为受控的身份切换提供基础：进程可以保留身份来源、使用当前有效身份进行权限检查，并在允许的范围内恢复先前保存的身份。

StarryOS 基线中的部分身份设置接口只返回成功，并未保存身份变化；调用者看似完成了切换，后续权限检查却仍在使用旧身份。MossOS 在 `ProcessData` 中保存六个 UID/GID 字段，并在 `setresuid`、`setresgid` 等接口中检查允许的转换范围：

```
let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}
// 分别更新三个字段，UID_GID_NO_CHANGE 表示不修改
if ruid != UID_GID_NO_CHANGE { proc_data.set_real_uid(ruid); }
if euid != UID_GID_NO_CHANGE { proc_data.set_euid(euid); }
if suid != UID_GID_NO_CHANGE { proc_data.set_suid(suid); }
```

代码 2-6　UID 三元组的保存与受限转换

在 `setreuid`、`setresuid` 及对应 GID 接口中，非特权进程只能在已有身份范围内切换，effective UID 为 0 的进程才可设置新值。effective UID/GID 改变后，执行权限、信号发送和 ptrace 等检查随之改变。目前 `setuid` 和 `setgid` 路径的权限约束仍较简化。

capability 把传统上集中在 root 身上的权限拆成多项能力。MossOS 通过按 PID 索引的状态表保存 effective、permitted、inheritable 和 bounding 集合，`capset` 会检查集合之间的基本约束。当前这些状态尚未与进程创建、`exec` 和 UID 变化完整联动，因此属于兼容性实现。

2.8.2　ptrace 最小控制链路

MossOS 新增 ptrace 的基础状态管理，建立 `TRACEME`、`ATTACH`、`DETACH`、停止、继续和 wait 报告之间的关系。attach 时先检查目标是否存在、是否已被跟踪以及调用者是否具有相应权限；目标停止后，跟踪者通过 wait 获取事件。

但是当前实现只能完成最小控制流程。`PTRACE_SYSCALL` 和 `PTRACE_SINGLESTEP` 暂按普通继续处理，寄存器、目标内存和 siginfo 读写请求返回 `EIO`，因此还不能视为完整的 GDB 后端。

2.9 本章小结

MossOS 选择沿用 StarryOS 的进程线程框架，围绕四条主线完成加固：以 `Thread` 和 `ProcessData` 组成双层对象模型，以线程归属、父子树和进程组会话维护三类组织关系，以 real、effective、saved ID 构成三元凭据，并把 clone、exec、exit、wait 串成完整生命周期。

在此基础上，MossOS 补充了 clone 参数校验和失败回滚，增加 `execveat`，完善子进程状态、等待接口和退出清理，并加入有状态的权限和 ptrace 控制。当前已实现 `vfork` 的父子同步和线程组组长发起的多线程 `exec` 收敛；非组长线程的 `exec`、完整终端作业控制、namespace 隔离和 ptrace 数据访问仍未完成。

第 3 章 调度与同步机制

任务等待事件的过程看起来直接：检查条件、登记唤醒入口、进入阻塞，事件到了再恢复。但如果通知恰好在“条件检查完、Waker 还没登记”这个窗口里到达时，等待方错过了这次通知，之后就没有人再来唤醒它了。本章大多数改动都对准这个竞争窗口，以及由此派生出的几个具体问题：超时后旧等待项还留在队列里，子进程状态变化和父进程登记 Waker 之间有空隙，Futex 通知与入队之间存在同样的竞争。

MossOS 沿用 StarryOS 的 RR 调度框架，在此基础上调整多核任务放置，并修正阻塞与唤醒间的竞争。任务等待事件时由 `Running` 进入 `Blocked`；事件到来后先恢复为 `Ready`，经调度再次运行时才重新检查等待条件。

3.1 概述

StarryOS 已经提供了可运行的任务调度骨架：`axtask` 管理任务状态和运行队列，`axsched` 用 RR 调度器按时间片轮转，多核下每个 CPU 有独立就绪队列，Futex 的 WAIT/WAKE、位掩码唤醒、重新排队和 robust list 也已实现，定时器与子进程等待接口也在。这部分 MossOS 主要是在 StarryOS 的基础上进行补充的。

问题集中在事件交错时的状态处理上。Futex 等待项只保存了 Waker 和位掩码，没有独立的通知状态，超时或被信号中断后旧记录还留在队列里——内核说不清通知究竟有没有到达，失效等待项也会占用后续唤醒名额。`FUTEX_WAIT_BITSET` 的绝对截止时间被当相对时长用，定时器重新设置后旧事件照样留在全局队列。子进程状态变化与父进程登记 Waker 之间，也同样有一段通知可能悄悄溜走的窗口。

MossOS 保留 RR 调度框架，同步路径的改动集中在三个环节：等待者如何登记，通知如何与阻塞状态配合，等待结束后如何清理记录。对应关系如下：

内容	StarryOS 基线	MossOS 的增量
任务调度	任务状态、RR 时间片和 CPU 亲和性	修正亲和性解析，按运行队列负载选择 CPU
阻塞唤醒	Future、Waker 和等待队列	调整 PollSet 唤醒过程，补充条件复查和等待项清理
定时处理	基础休眠和进程定时器	区分相对与绝对时间，使用 TimerKey 过滤旧事件
Futex	WAIT、WAKE、位掩码和重新排队	加入独立通知状态，修正超时语义和退出清理

表 3-1　调度与同步机制的主要增量

这些修改共同作用于任务离开 CPU 和重新获得 CPU 的过程。其中最难处理的是通知与状态转换可能同时发生：任务检查完条件、尚未登记 Waker 时，事件可能已经到达；任务准备进入阻塞时，通知也可能先一步发生。下面的叙述会先用任务状态、抢占和上下文切换说明调度基础，再围绕这一竞争窗口讨论通用阻塞唤醒机制，最后分析定时等待、子进程等待和 Futex 的具体处理。

3.2 任务调度机制

3.2.1 任务状态与状态转换

任务调度围绕四种状态展开。调度器不需要理解任务正在等待哪一种资源，只需判断任务当前属于以下四种中的哪种状态：

```
pub enum TaskState {
    Running = 1, // 正在 CPU 上执行
    Ready   = 2, // 可以运行，正在队列中等待
    Blocked = 3, // 正在等待事件，暂时不能运行
    Exited  = 4, // 已经结束，等待资源回收
}

pub struct TaskInner {
    // 调度器通过该字段判断任务当前能否运行
    state: AtomicU8,
    // ...
}
```

代码 3-1　任务状态的数据结构

`state` 使用原子整数保存任务状态。定时等待、Futex 和子进程等待各自维护不同的业务条件，但要重新获得 CPU，都必须让任务从 `Blocked` 回到 `Ready`。调度器只处理这一状态转换，并不判断具体等待条件是否已经满足。四种状态之间的转换关系如图所示。

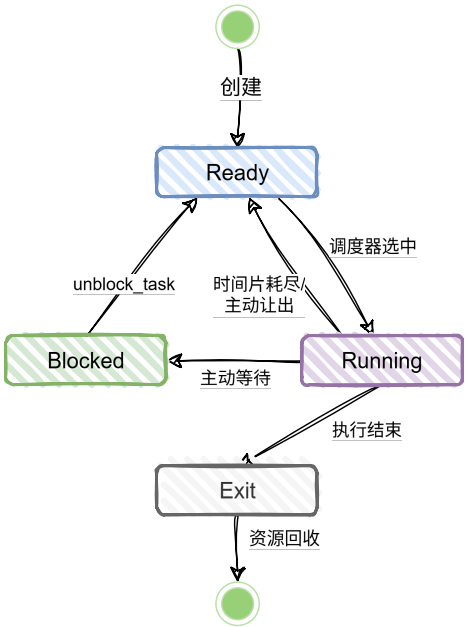


图 3-1　TaskState 的状态转换关系

新任务先以 `Ready` 状态进入就绪队列，被调度器选中后变为 `Running`。主动让出 CPU 的任务仍具备运行条件，因此会回到 `Ready`；等待资源的任务则进入 `Blocked`，由 `unblock_task` 恢复为 `Ready`。任务执行结束后进入 `Exited`，并在资源回收后离开状态机。MossOS 的定时等待、Futex 和子进程等待都通过这组状态转换重新接入调度器。

状态修改通过原子状态和运行队列中的唤醒检查配合完成。多个通知同时到达时，唤醒路径应该只允许仍处于 `Blocked` 的任务重新进入就绪队列。

3.2.2 时间片轮转与抢占

任务按照 RR 策略轮流使用 CPU。单核环境下只有一个运行队列，所有就绪任务都在该队列中轮转。多核环境下，每个 CPU 各自维护 RR 队列，调度器只在本 CPU 队列内选择下一个任务，因此 RR 轮转是每 CPU 局部的，而不是所有 CPU 共享一个全局队列。任务的默认时间片是 5 个时钟滴答；时间片用完后，任务回到所在队列的队尾。

时钟中断每到来一次，`scheduler_timer_tick`（调度器的时钟节拍处理函数）就更新当前任务的时间片。计数归零时，内核先设置待抢占标志，而不是立刻在中断处理或持锁位置切换任务。等代码离开禁止抢占的临界区后，再执行真正的重新调度。

时钟中断 → 时间片减一 → 设置待抢占标志 → 离开临界区 → 重新调度

时间片归零后，内核先记录抢占请求，等执行离开禁止抢占的临界区再调度。任务如果在修改运行队列、处理中断或持有关键锁时被换出，其他 CPU 可能看到尚未完成的中间状态。延迟抢占把上下文切换放到状态一致的位置，运行队列和锁的处理也能先行结束。

如果当前没有普通任务可运行，CPU 会进入 idle 任务（没有普通任务时运行的空闲任务），等待新的任务或中断到来。

3.2.3 任务选择与上下文切换

前两节已经说明了任务状态如何变化，以及时间片耗尽后如何发起抢占。本节从重新调度真正开始的位置继续：当前任务的状态处理完毕后，`resched` 从本 CPU 的就绪队列中选出下一个任务；如果队列为空，就让 idle 任务暂时接管 CPU。

```rust
fn resched(&mut self) {
    // 从就绪队列选择任务；队列为空时运行 idle 任务
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将选中的任务设为正在运行
    next.set_state(TaskState::Running);

    // 保存当前任务现场，恢复下一个任务现场
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

代码 3-2　从选择任务到切换现场的主要流程

选出任务后，`switch_to` 先将它标记为 `Running`，再切换任务现场。这里的“现场”是任务暂停前的运行位置。每个任务都有一个 `TaskContext`，其中保存内核栈指针、需要保留的寄存器和返回位置。切换时，当前 CPU 的这些值被保存到旧任务中，再从新任务的 `TaskContext` 恢复。旧任务以后再次被选中时，就能接着原来的位置运行。

`TaskContext` 是 `TaskInner` 中的独立字段，其中的栈指针指向任务内核栈。`switch_to` 把 prev 的现场写入对应的 `TaskContext`，再从 next 恢复。用户任务的页表根也保存在其中：RISC-V 使用 `satp`，LoongArch 使用 `pgdl`。用户态寄存器则保存在 `UserContext` 中。至此，从 3.2.2 的抢占请求到完成切换，可以连成下面这条流程：

设置待抢占标志 → 到达安全调度位置 → 选择下一任务 → 保存旧现场 → 恢复新现场

3.2.4 CPU 亲和性与调度接口

每个任务都保存一份 CPU 亲和性掩码 `cpumask`，用来限制它可以在哪些 CPU 上运行。例如，掩码只包含 CPU 0 和 CPU 2 时，调度器仅会在这两个 CPU 的运行队列中放置该任务。

选核和迁移只在多核配置下发生。单核配置的 `select_run_queue` 始终返回唯一的运行队列，亲和性只能包含 CPU 0，也不需要负载均衡或跨 CPU 迁移。下述低负载选核、原 CPU 恢复和负载再平衡均属于多核路径。

新任务默认可以使用所有已经启用的 CPU。初次放置时，调度器在 `cpumask` 允许且已初始化的 CPU 中选择负载最低的运行队列；就绪任务数和当前运行的非 idle 任务共同计入负载，负载相同时再用轮转起点打破平局。

```
let mut selected = None;
let mut minimum_load = usize::MAX;
let cpu_num = active_cpu_num();
let start = RUN_QUEUE_INDEX.fetch_add(1, Ordering::Relaxed) % cpu_num;
for offset in 0..cpu_num {
    let cpu_id = (start + offset) % cpu_num;
    if cpumask.get(cpu_id)
        && RUN_QUEUE_READY[cpu_id].load(Ordering::Acquire)
    {
        let load = unsafe {
            RUN_QUEUES[cpu_id].assume_init_ref().load()
        };
        if load < minimum_load {
            selected = Some(cpu_id);
            minimum_load = load;
        }
    }
}
```

代码 3-3　在亲和性范围内选择低负载 CPU

已经运行过的任务被唤醒时优先回到原 CPU，除非新的亲和性已排除该 CPU。用户任务返回用户态前，内核还会在不持有 CPU 局部状态的安全位置检查负载；只有当当前 CPU 比目标 CPU 至少多两个任务时才执行迁移。

用户传入的亲和性范围是一张位图，每一位对应一个 CPU。MossOS 修正了这张位图的解析过程：内核按照实际配置的 CPU 数量读取内容，再把其中被置为 1 的位置写入任务掩码。用户程序通过 Linux 的 `sched_getaffinity` 查询范围，通过 `sched_setaffinity` 修改范围。

```
// 用户位图中的每一位对应一个 CPU
for cpu_id in 0..axtask::active_cpu_count() {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
```

```
            cpu_mask.set(cpu_id, true);
        }
    }

    if task.id() == current().id() {
        // 当前 CPU 被排除时立即迁移当前任务
        axtask::set_current_affinity(cpu_mask);
    } else {
        // 其他任务只更新掩码
        task.set_cpumask(cpu_mask);
    }
```

代码 3-4　CPU 亲和性掩码的设置与任务迁移

修改当前任务的掩码时，若当前 CPU 已被排除，MossOS 会立即将其迁移到允许的运行队列。修改其他任务时只更新 `cpumask`，不会立即迁移正在运行的目标任务。

用户调用 `sched_yield` 主动让出 CPU 后，MossOS 将当前任务放回就绪队列，再由调度器选择其他任务运行。这个接口适合任务暂时没有紧急工作、希望让其他就绪任务先执行的情况。

3.3　阻塞与唤醒机制

MossOS 延续 StarryOS 的 Future/Waker 模型。这套机制的实际好处在于：timerfd、pidfd、Futex 等来源各不相同的等待，都能挂到同一套唤醒路径上，`block_on` 也因此能以统一方式暂停和恢复任务，不需要为每类等待单独实现阻塞逻辑。代价是每个等待路径都得正确维护通知状态和双重检查，否则竞争窗口就会漏掉通知。这正是本节要讨论的。

3.3.1　事件等待的基本流程

调度器只知道任务处于 `Blocked`，并不知道任务在等什么。事件对象和等待者之间还需要一套通知协议。MossOS 沿用 Future 表示”尚未完成、之后还要继续检查的操作”，用 Waker 记录”条件可能变化时如何通知当前任务”。

以读取管道为例，任务先检查缓冲区。已有数据就直接读取；没有数据则把自己的 Waker 登记到管道中，再由 Future 返回“暂时未完成”。另一个任务写入数据后，管道调用保存的 Waker，通知读任务继续检查。

检查条件 → 保存 Waker → 再次检查 → 等待通知

事件发生 → 调用 Waker → Future 再次执行 → 重新检查条件

图 3-2　事件登记与通知的基本流程

关键步骤是登记 Waker 后的第二次条件检查。假如数据恰好在第一次检查结束后到达，此时写入方还找不到等待者，无法调用 Waker。登记完成后立刻复查缓冲区，读任务仍能看到已经到达的数据，不会因为漏掉那次通知而一直等待。

收到通知后还要重新检查条件。Waker 只表示“情况可能有变化”，并不保证资源一定属于当前任务。例如多个读任务同时被通知时，管道中的数据可能已经被其中一个任务取走，其他任务仍需继续等待。

3.3.2 Waker 与 PollSet

Waker 是 StarryOS 留给事件对象的任务唤醒入口。事件对象不必直接操作运行队列，只需保存等待者提供的 Waker，并在状态发生变化时调用 `wake()`。MossOS 继续使用这一接口，把 timerfd、pidfd 和 Futex 等等待路径接到同一套任务唤醒流程上。

`PollSet` 管理一组 Waker，管道、eventfd、timerfd、pidfd 和终端等对象都通过它登记等待者。MossOS 增加 `has_waiters`，没有等待者时直接返回；真正唤醒时，先在锁内取出并清空本轮 Waker，再到锁外逐个调用。内部锁只保护集合操作，不覆盖可能继续触发调度的 `wake()`。

```
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者时，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本轮登记的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正唤醒时已经不再持有内部锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

代码 3-5　PollSet 取出并唤醒等待者

`take_wakers` 在取出 Waker 的同时清空集合，因此后续事件不会再次使用这一批旧记录。

PollSet 只负责“有人需要重新检查了”，不会替业务对象判断条件是否真的满足。管道仍要检查缓冲区，子进程等待仍要检查状态，Futex 也要检查通知标记。

3.3.3 block_on 与通知竞争处理

Future 通过反复轮询判断操作是否完成，而同步系统调用需要在结果尚未产生时暂停当前任务。`block_on` 连接这两种执行方式：Future 返回 `Ready` 时，系统调用取得结果；返回 `Pending` 时，当前任务准备阻塞，把 CPU 交给其他任务。任务选择和现场切换仍由 3.2.3 的调度路径完成。

```
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先记录通知已经到达
        *self.woke.lock() = true;
        // 再把阻塞任务放回合适的运行队列
        select_run_queue(&task).unblock_task(task, false);
    }
}
```

```
match future.poll(&mut context) {
    Poll::Pending => {
        if !*woke {
            run_queue.blocked_resched(woke);
        } else {
            // 通知先到，清除本次边沿后立即重新轮询
            *woke = false;
            drop(woke);
        }
    }
    Poll::Ready(output) => return output,
}
```

代码 3-6　Waker 与 block_on 的阻塞唤醒配合

Future 在轮询中登记 Waker 并返回 `Pending` 后，通知可能在任务切换为 `Blocked` 之前到达。Waker 先把 `woke` 设为真，再尝试唤醒任务。若通知先到，任务此时仍为 `Running`，`unblock_task` 不改变状态；`block_on` 看到 `woke` 后不再阻塞，而是清除本次通知并立即重新轮询。若阻塞先完成，`unblock_task` 则把任务从 `Blocked` 改为 `Ready`。

单核下，唤醒后的任务直接回到同一运行队列；Futex 唤醒到等待者时，当前任务会主动让出 CPU，以便被唤醒任务及时运行。多核下，等待者通常回到原 CPU 的运行队列，唤醒方无需让出当前 CPU；若目标 CPU 处于 idle，启用 IPI 时内核会向其发送合并后的唤醒 IPI。

正常事件之外，信号也可以结束等待。`interruptible` 同时检查原来的 Future 和任务的信号状态。收到未屏蔽信号后，等待提前结束，上层系统调用再决定返回 `EINTR`、保存剩余时间或按信号规则重启。

等待因超时或信号提前结束后，还必须删除本轮登记。否则后续事件仍可能找到旧 Waker，既产生无效唤醒，也会干扰唤醒数量统计。PollSet 会清空已经取出的 Waker，Futex 则按等待项精确清理，具体过程将在 3.5 节说明。

3.4　通用阻塞机制的应用

3.3 节建立了”检查条件、登记 Waker、阻塞、收到通知后复查”的通用流程。本节以定时等待为例说明这套流程如何落地——业务条件是”截止时间是否到达”，任务恢复后仍要重新检查，不能把一次唤醒直接当作最终结果。子进程等待与信号机制紧密相关，通知路径在第六章说明。

3.4.1　定时等待与截止时间

定时等待先根据系统调用参数选择时间来源，再把等待目标统一换算为截止时间。单调时钟只向前推进，适合表示“再等待多久”；实时时钟对应系统日历时间，适合指定一个实际时刻。

时间参数分为相对时长和绝对时间。相对时长需要加到当前时间上，绝对时间本身就是截止时间；如果截止时间已经到达，等待立即结束。统一转换后，后续逻辑只需比较“当前时间是否达到截止时间”。同样的区分也用于修正后文 `FUTEX_WAIT_BITSET` 的超时语义。

`nanosleep` 使用单调时钟完成相对休眠。`clock_nanosleep` 还允许选择时钟，并通过 `TIMER_ABSTIME` 表示绝对时间。休眠期间如果收到未屏蔽信号，MossOS 会结束本次等待，向用户返回 `EINTR`，并写回尚未用完的时间。

3.4.2 定时器队列与到期处理

StarryOS 的定时事件可以按截止时间触发，但定时器重新设置后，旧事件可能仍留在队列中。仅凭截止时间无法判断取出的条目是否仍然有效。MossOS 改用 `BTreeMap` 保存定时器，并用截止时间 `deadline` 和递增序号 `seq` 组成唯一的 `TimerKey`。后台定时任务每次取出最早到期的条目，到期后更新定时器状态并发送相应信号。

```
struct TimerKey {
    // 定时事件应当触发的时刻
    deadline: Duration,
    // 区分截止时间相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    // 为新事件生成不重复的序号
    next_seq: u64,
    // 按到期顺序保存全部定时事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

代码 3-7　定时器队列的关键数据结构

同一时刻可能有多个定时器到期，`seq` 使每个事件都有独立的键。重新设置或取消定时器时，内核删除旧键；后台任务取出事件后，还要确认它与定时器当前保存的键一致。只有匹配的事件才会触发信号，重新设置前留下的旧条目会被忽略。新增更早的定时器时，系统还会通知后台任务重新计算下一次等待时间。

每个线程还有一个时间管理对象 `TimeManager`，用来分别累计用户态和内核态运行时间。实际时间定时器 `ITIMER_REAL` 按经过时间发送 `SIGALRM`；用户时间定时器 `ITIMER_VIRTUAL` 只统计用户态时间；性能分析定时器 `ITIMER_PROF` 同时统计用户态与内核态时间。周期定时器到期后重新装载，一次性定时器到期后停止。

3.5 Futex 同步机制

3.5.1 基本思想与数据结构

Futex 把无竞争的同步留在用户态。线程先通过原子变量尝试获得锁，只有发现资源被占用时才进入内核等待。因此，大多数没有竞争的加锁和解锁不需要系统调用。

进入内核后，MossOS 使用 `FutexKey` 定位等待队列。进程私有 Futex 以用户地址为键；共享匿名映射和文件映射还要结合共享后端与区域偏移，保证不同进程即使把同一页映射到不同虚拟地址，也能找到同一个 Futex。

每个 Futex 表项保存等待队列和 `owner_dead` 状态。为了分清正常唤醒、超时和信号中断，MossOS 将单个等待项组织为：

```
pub struct WaitQueue {
    // 每个等待项依次保存：
    // 通知状态、唤醒位掩码和任务唤醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
```

```
    >,
}
```

代码 3-8　Futex 等待队列的数据结构

元组中的 `AtomicBool` 是 `notified`，记录等待项是否已经被选中唤醒；`u32` 保存 bitset；`PollSet` 保存任务的唤醒入口。业务状态与唤醒动作由此分开：`notified` 防止通知丢失或重复处理，PollSet 负责让任务重新参与调度。

图 3-3 展示了 FutexKey 如何定位 WaitQueue，以及每个等待项三个字段的分工——`notified` 记录通知结果，bitset 过滤唤醒目标，PollSet 持有唤醒入口。

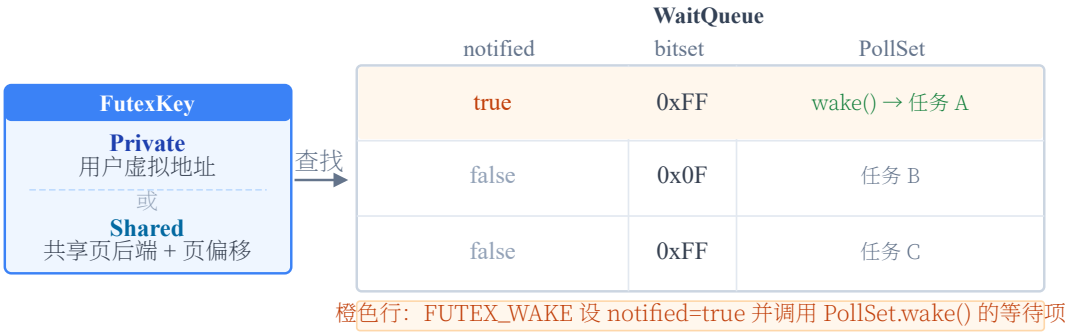


图 3-3　FutexKey、WaitQueue 与等待项三元组的结构关系

3.5.2　WAIT 与 WAKE

等待者只有在用户地址仍等于期望值时才应睡眠。系统调用入口先做一次快速检查，不相等就返回 `EAGAIN`；准备入队时，内核在持有等待队列锁的情况下再次读取用户值。第二次检查把用户值判断与入队操作放进同一个受保护区间，条件已经改变时，本次调用不再阻塞。

条件仍成立时，内核创建等待项并进入可中断、可超时的 Future。`FUTEX_WAKE` 从队列中选择指定数量的等待者，先把 `notified` 设为真，再调用 PollSet；被正常唤醒的等待项同时从队列删除，不会继续占用下一次 wake 的名额。

与 3.3.3 不同，这里额外处理 Futex 等待项入队后的竞争：`notified` 由 `FUTEX_WAKE` 写入，并在登记 PollSet 前后复查。它记录的是单个 Futex 等待项的通知结果，后续位掩码唤醒、超时清理和重新排队都会保留或处理该状态。

3.5.3　位掩码唤醒与重新排队

位掩码唤醒与重新排队也使用新增的 `notified` 状态和退出清理。`FUTEX_WAIT_BITSET` 为等待者附带位掩码，`FUTEX_WAKE_BITSET` 只唤醒 bitset 与目标 mask 有交集的任务，从而避免叫醒无关线程。

`FUTEX_REQUEUE` 先唤醒最多 `wake_count` 个任务，再把最多 `requeue_count` 个等待项移动到另一个 Futex 队列。`FUTEX_CMP_REQUEUE` 在此之前还会比较原地址的用户值，不匹配时返回 `EAGAIN`。移动过程中保留 `notified`、bitset 和 PollSet，不重新登记 Waker。

这类操作常用于 pthread 条件变量。广播时不必把所有线程同时叫醒争抢互斥量，可以先唤醒少量线程，再按指定数量把等待者转移到互斥量对应的队列。如 图 3-4 所示，未处理的等待者仍留在原队列，被转移的等待项保留原有的 `notified`、bitset 和 PollSet。

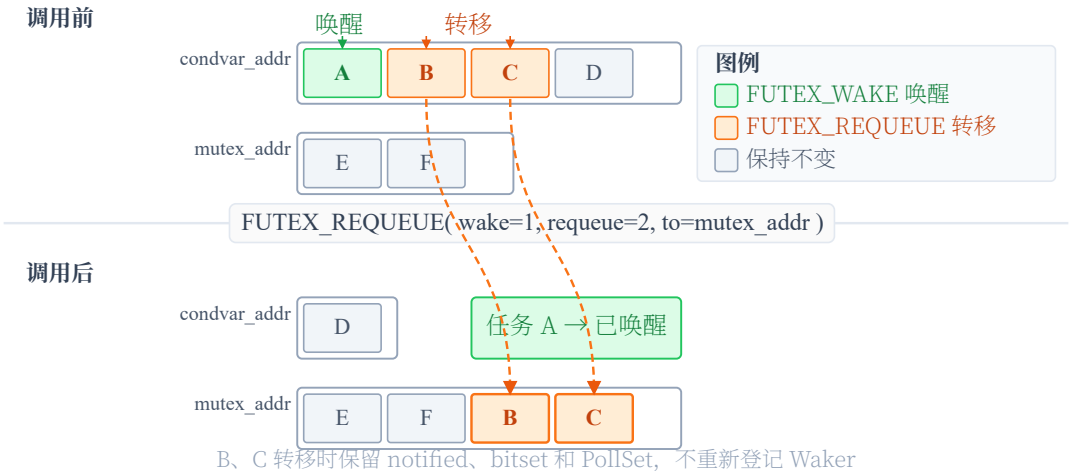


图 3-4　FUTEX_REQUEUE 唤醒和转移指定数量的等待者

3.5.4　超时、中断与等待修正

Futex 等待可能正常收到通知，也可能先超时或被信号打断。MossOS 用 `interruptible(timeout(...))` 组合这三条路径。超时返回 `ETIMEDOUT`，信号中断返回 `EINTR`；两种提前结束都会按照 `notified` 的对象地址精确删除当前等待项。

原有实现没有把 `FUTEX_WAIT` 和 `FUTEX_WAIT_BITSET` 的时间语义分开。MossOS 按 Linux 规则处理：前者接收相对时长，后者接收绝对截止时间，并根据 `FUTEX_CLOCK_REALTIME` 选择实时时钟或单调时钟。

独立通知状态解决了“通知是否已经到达”的判断问题，前后复查覆盖登记窗口，退出清理则删除已经失效的等待项。三者分别处理通知丢失、重复处理和队列残留。相关修改修复了 LoongArch64 上 `futex_wait`、pthread cancellation 和部分休眠测试卡住的问题。

3.5.5　robust list 与线程退出清理

线程持有互斥量时如果直接退出，其他线程可能永远等不到解锁，从而造成死锁。针对这种情况，MossOS 完善了退出时的 Futex 清理。用户态线程库通过 `set_robust_list` 把 robust list 头指针交给内核，链表记录该线程可能持有的 Futex。

线程退出时，MossOS 按 `futex_offset` 找到对应 Futex，将 `owner_dead` 设为真，并唤醒一个等待者。被唤醒线程收到 `EOWNERDEAD` 后，由用户态线程库检查共享数据并恢复锁的一致性。链表遍历受 `ROBUST_LIST_LIMIT` 限制，损坏的用户链表不会让内核无限循环。

线程退出还会清零 `clear_child_tid`，并唤醒这个地址对应的 Futex。这条路径支持 `pthread_join` 等线程回收操作。

3.6　本章小结

StarryOS 已经具备 RR 调度、任务状态转换、上下文切换和基础事件等待，MossOS 没有另建一套调度器，而是改进多核运行队列的负载选择和亲和性迁移。阻塞唤醒方面的修改集中在等待者登记、通知状态、恢复后的条件复查以及失效记录清理。

PollSet 缩短了持锁唤醒路径，`block_on` 用 `woke` 协调通知与阻塞，定时器用 `TimerKey` 识别仍然有效的事件，Futex 则通过 `notified`、双重检查和精确删除处理竞争。条件不满足时任务进入阻塞；条件可能变化时，唤醒路径先将其恢复为就绪状态，调度运行后再复查条件，未满足则继续阻塞。

第 4 章 内存管理

4.1 概述

StarryOS 已经具备用户虚拟内存管理的主体框架，其核心结构如 代码 4-1 所示。`AddrSpace` 记录可用地址范围和当前页表；`MemoryArea` 按起始地址组织互不重叠的虚拟区域，每个区域绑定一个 `Backend` 决定页面来源；`Backend` 封装四种映射后端。物理页在缺页时按需装入，`fork` 通过 `AddrSpace::try_clone` 复制区域描述，并对私有可写页面执行 COW 保护。

```
AddrSpace
├── va_range              可用虚拟地址上下界
├── areas: MemorySet
│   └── BTreeMap<VirtAddr, MemoryArea>
│       ├── va_range      连续的虚拟地址范围
│       ├── flags         读 / 写 / 执行 / 用户态权限
│       └── backend       Linear | Cow | Shared | File
└── pt: PageTable         已建立的虚拟页 → 物理页映射
```

代码 4-1　AddrSpace 三层结构概览

MossOS 沿用这套框架，主要补充了 `mmap` 的参数校验、映射元数据和部分配套语义。当前 `sys_mmap` 能处理 `MAP_PRIVATE`、`MAP_SHARED` 和 `MAP_ANONYMOUS` 的基本组合，对应关系如 表 4-1 所示。`madvise`、`mremap` 和 `mlock` 等接口也增加了必要的检查与处理，但部分标志仍属于兼容性实现，例如 `mlock` 尚未真正固定物理页。

映射请求	**Backend**	行为
`MAP_PRIVATE` `MAP_ANONYMOUS`	Cow	建立私有匿名区域，首次访问时清零分配
`MAP_PRIVATE` 文件映射	Cow	从文件读取初始内容，写入后仅修改私有副本
`MAP_SHARED` `MAP_ANONYMOUS`	Shared	按需分配物理页，多进程共享同一组页面
`MAP_SHARED` 文件映射	File	通过文件缓存访问，多映射看到同一份内容
设备物理范围	Linear	按固定偏移直接映射设备提供的物理地址

表 4-1　mmap 标志与 Backend 的对应关系（StarryOS 基线已具备）

表 4-2 总结了 MossOS 当前实现的主要机制。

机制	基线状态	MossOS 的处理
栈自动增长	`MAP_GROWSDOWN` 已定义但忽略	在映射起点下方的预留窗口内，按缺页地址补充 4 KiB 匿名页
mremap	仅按匿名映射重建	支持收缩、原地扩展和迁移；`MREMAP_FIXED` 拒绝新旧范围重叠
WIPEONFORK	不存在	fork 后在子进程中清零标记的私有匿名区域
/proc/[pid]/maps	不存在	输出地址范围、访问权限以及私有或共享属性
PROT_WRITE	W=1,R=0 无法形成合法的 RISC-V 页表项	硬件权限补 READ，用户可见权限保持不变
缺页信号	未区分失败原因	根据地址是否仍属于可访问 VMA 选择 SIGBUS 或 SIGSEGV

表 4-2　MossOS 在 mmap 上的语义补全

下面逐项展开这些机制。

4.2 栈自动增长

用户栈可能因大型局部变量、深层递归或 `alloca` 超出初始映射范围。`MAP_GROWSDOWN` 用于标记可向低地址增长的区域，使这类访问不必立即触发 `SIGSEGV`。

代码 4-2 展示了当前处理路径。建立 GROWSDOWN 映射时，内核在映射起点下方记录一个最多 256 页的增长窗口。普通缺页处理失败后，`try_growdown_fault` 按 pid 查找窗口；若缺页页地址位于 `[low, high)` 且窗口未被阻断，就以 Cow Backend 建立一个清零的 4 KiB 匿名页。当前实现不修改原 `MemoryArea` 的起始地址，也不要求新增页与已有映射连续。

```
handle_page_fault(vaddr)
├── areas.find(vaddr) 命中 → 正常 Backend::populate
└── 未命中
    └── try_growdown_fault(vaddr)
        ├── 查找当前 pid 的增长窗口
        ├── 页地址在 [low, high) 且窗口未阻断?
        │   ├── 是 → 建立一个 4 KiB 私有匿名页
        │   └── 否 → SIGSEGV
        └── MAP_FIXED 与窗口冲突 → 标记窗口为 blocked
```

代码 4-2　GROWSDOWN 缺页处理流程

增长范围由固定的 `low` 下界限制，窗口外访问仍触发 `SIGSEGV`。若后续 `MAP_FIXED` 映射与窗口冲突，`block_growdown` 会禁止该窗口继续补页，避免增长区域侵入其他映射。

4.3 mremap 迁移

当前 `mremap` 支持三条路径：缩小时解除尾部映射；后方空间充足时原地扩展；无法原地扩展且设置 `MREMAP_MAYMOVE` 时，建立新的私有匿名映射、复制原有数据，再解除旧映射。未设置 `MREMAP_MAYMOVE` 且空间不足时返回 `ENOMEM`。

MREMAP_FIXED 要求同时设置 MREMAP_MAYMOVE，并要求目标地址页对齐。为避免复制和解除映射时相互覆盖，只要源范围与目标范围重叠，当前实现就返回 EINVAL。迁移和扩展路径尚未完整保留文件映射、共享属性及原 Backend 类型，因此不能视为完整的 Linux mremap 语义。

4.4 MADV_WIPEONFORK

madvise(MADV_WIPEONFORK) 指示内核在 fork 后清零子进程中的指定区域，可用于避免子进程继承运行库状态、同步数据或敏感信息。当前实现仅允许它作用于私有匿名映射；其他映射类型返回 EINVAL。

MossOS 用全局 WIPEONFORK_AREAS 表按 pid 和地址范围记录标记。fork 时，inherit_wipeonfork_ranges 在子地址空间中逐段写零；写入会触发 COW，因此父进程内容不受影响。标记随后继承给子进程，MADV_KEEPONFORK 则移除指定范围的标记。

4.5 /proc/[pid]/maps

/proc/[pid]/maps 为调试和运行时探测提供进程的基本内存布局。mmap_proc_maps() 遍历地址空间中的 MemoryArea，输出起止地址、r/w/x 权限以及 p/s 私有或共享属性。权限和映射类型直接来自每个 VMA 的 VmaMetadata，mprotect 修改权限时也会同步更新该元数据。

当前偏移、设备号和 inode 仍输出固定占位值，也不输出文件路径，因此这里只提供基础映射列表，尚不等同于 Linux 的完整 /proc/[pid]/maps。

4.6 PROT_WRITE 与 SIGBUS

RISC-V 将 W=1、R=0 的页表项编码保留，因而不能直接建立硬件层面的“只写”页面。用户以 PROT_WRITE（不含 PROT_READ）调用 mmap 时，内核仍需生成合法的页表权限。

MossOS 在 MmapProt 到 MappingFlags 的转换中进行补偿：只要设置 WRITE，硬件映射就同时设置 READ | WRITE；VMA 元数据仍保存用户请求的权限，因此 /proc/[pid]/maps 仍可显示只写权限。

缺页处理还区分地址错误和后端错误：若填页失败，但地址仍位于权限允许的 VMA 内，说明虚拟地址有效而后端无法提供页面，此时发送 SIGBUS；地址不属于可访问 VMA 时发送 SIGSEGV。此外，SIGBUS_AREAS 目前只为共享文件映射的特定 mremap 路径记录写缺页范围，并不是通用的文件尾追踪机制。进程退出时会统一清理这些辅助记录。

4.7 本章小结

MossOS 在既有地址空间框架上补充了多项 mmap 配套语义：GROWSDOWN 在受限窗口内处理栈增长缺页，mremap 支持收缩、原地扩展和非重叠迁移，WIPEONFORK 在子进程中清零指定私有匿名区域，VMA 元数据为基础版 /proc/[pid]/maps 提供权限和映射类型。PROT_WRITE 转换适配了 RISC-V 页表约束，缺页入口也能依据地址是否仍属于有效 VMA 区分 SIGBUS 与 SIGSEGV。尚未完整实现的映射属性保留、maps 文件信息和页面锁定语义则作为当前限制保留。

第 5 章 文件系统

5.1 概述

从 Unix 的设计哲学 —— “Everything is a file.”出发，文件系统要解决的核心问题是：让用户程序用同一套接口去访问性质完全不同的对象——磁盘上的 ext4 文件、内存里的临时文件、/proc 下的内核信息、/dev 下的设备节点。

StarryOS 为这个问题提供了一套完整的基础框架。分四层：axfs-ng-vfs 定义文件系统、目录节点、文件节点的接口；axfs 实现路径解析、页缓存和 ext4/FAT32 后端；kernel/src/file/ 将 VFS 对象统一进文件描述符表；syscall/fs/ 把系统调用参数翻译为上层调用。

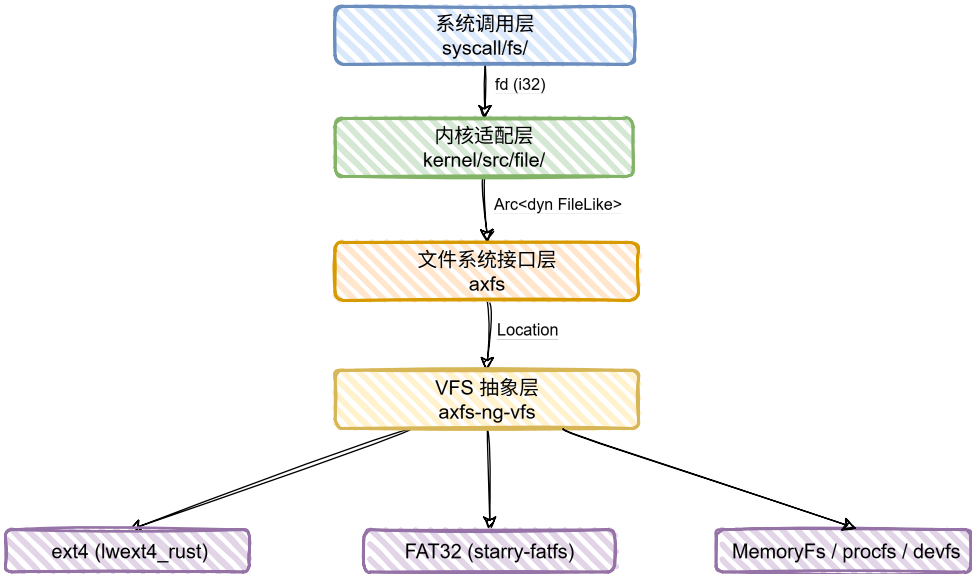


图 5-1　文件系统分层架构

在这套框架中，StarryOS 已经提供了几个关键设计：

- **Location**：将目录项与所属挂载点绑定为统一句柄，路径解析穿越挂载点时自动切换文件系统；
- **Mountpoint** 树：子节点用 `Weak` 引用，卸载时自动释放，跨设备检测用 `Arc::ptr_eq` 对比挂载点指针；
- **NodeFlags**：四个标志位（`STREAM`、`NON_CACHEABLE`、`ALWAYS_CACHE`、`BLOCKING`）让节点声明自身 I/O 契约，上层无需判断节点类型；
- **FsContext** + 路径解析：记录任务的 `root_dir` 和 `current_dir`，创建进程时默认复制，在 `CLONE_FS` 下共享；四个解析变体覆盖不同 syscall 语义，符号链接循环检测上限为 40 层；
- 文件系统后端：lwext4_rust 接入 ext4，starry-fatfs 接入 FAT32，MemoryFs 提供内存文件系统，SimpleFs 框架支撑 procfs/devfs。

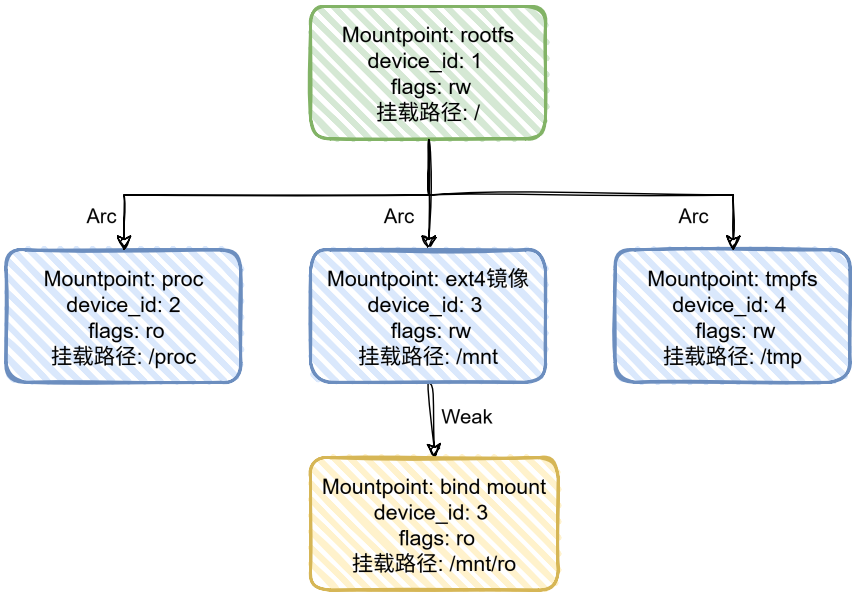


图 5-2　Mountpoint 树

MossOS 在此基础上完善了传统与新式挂载接口，引入 inode 元数据缓存和 Folio 文件页缓存，并补充扩展属性、inotify、fanotify 等功能。下面按这些方向展开。

5.2　挂载子系统

挂载解决的核心问题是：把独立的文件系统拼到同一棵目录树下。`mount` 调用在 `Mountpoint` 树上添加节点，路径解析穿越目录时自动切换到对应文件系统的根——这个设计来自 StarryOS。StarryOS 的 `sys_mount` 当前支持 `tmpfs`，flags 参数尚未处理；`sys_umount2` 同样未包含权限和状态检查。MossOS 在此基础上扩展了挂载操作的类型和语义。

5.2.1　传统挂载接口的完善

`sys_mount` 在 MossOS 中除了普通挂载外，支持了三种改变已有挂载关系的操作模式：

标志	含义
`MS_BIND`	bind mount：让已有目录树在另一条路径下可见，直接复用源挂载点对象，不创建新文件系统实例
`MS_MOVE`	迁移挂载点：先在目标路径建立 bind mount，再卸载源挂载
`MS_REMOUNT`	修改现有挂载的标志（如读写→只读），修改前检查是否有打开的写文件

表 5-1　sys_mount 新增的操作模式

此外，MossOS 增加了权限检查（要求 euid == 0），支持 10 种挂载标志（`MS_RDONLY`、`MS_NODEV`、`MS_NOEXEC`、`MS_NOSUID`、`MS_NOATIME`、`MS_NODIRATIME`、`MS_RELATIME`、`MS_STRICTATIME`、`MS_REMOUNT`、`MS_NOSYMFOLLOW`）。普通磁盘和 Loop 设备上的 ext2、ext3、ext4 统一由 ext4 后端解析；`xfs`、`btrfs`、`vfat`、`exfat`、`ntfs` 等类型目前用于兼容测试中的挂载流程，挂载后提供隔离的内存文件系统，不解析对应的磁盘格式。

`sys_umount2` 同样做了大量补全：

标志 / 行为	说明
MNT_DETACH	延迟卸载：跳过打开写文件的 busy 检查，并从挂载树中移除挂载
MNT_EXPIRE	两阶段过期：首次调用标记并返回 EAGAIN；其间未再次访问时，第二次调用执行卸载
UMOUNT_NOFOLLOW	不跟随符号链接，直接操作路径本身
卸载前清理	移除页缓存（drop_file_caches_by_device）、刷新文件系统脏数据（flush）、发送 inotify IN_UNMOUNT 事件
忙碌检查	MNT_DETACH 未设置时检查是否有打开的写文件，有则返回 EBUSY

表 5-2　sys_umount2 新增功能

5.2.2 新式挂载 API

Linux 新式挂载 API 将文件系统配置、实例化和接入目录树拆开。StarryOS 没有实现这套接口，MossOS 补充了其主要调用链。

```
fd = fsopen("ext4")                          // 创建文件系统上下文
fsconfig(fd, "source", "/dev/loop0")  // 配置设备路径
fsconfig(fd, "cmd", "create")           // 完成配置，准备挂载
mfd = fsmount(fd, 0, 0)                       // 实例化为挂载对象 fd
move_mount(mfd, "", "/mnt", MOVE_MOUNT_F_EMPTY_PATH)
```

代码 5-1　新式挂载 API 的主要调用流程

当前实现覆盖以下职责：

- fsopen：按文件系统类型名创建上下文，返回 FsContextFd；
- fsconfig：设置 source 路径并处理 FSCONFIG_CMD_CREATE；其余部分命令仅完成参数兼容，尚未保存为文件系统配置；
- fsmount：将配置完成的上下文实例化为 DetachedMountFd，其中只有只读属性会转换为实际挂载标志；
- open_tree：打开已挂载的文件系统树作为 detached mount fd（OPEN_TREE_CLONE 模式），为 bind mount 提供基于 fd 的接口；
- move_mount：在 MOVE_MOUNT_F_EMPTY_PATH 模式下将 detached mount fd 接入目标路径，尚未支持路径到路径的移动。

DetachedMountFd 是一个关键设计——它在挂载操作期间作为“尚未附加的挂载对象”存在，可以是全新文件系统（Filesystem { fs, mount_flags }）或已有挂载树的引用（Bind { source: Location }）。这个中间态让配置和挂载的分离成为可能。

5.2.3 Loop 设备

LTP 文件系统测试中大量用例需要在运行时把镜像文件（如 ext4.img）当块设备挂载：先 mkfs.ext4 格式化一个普通文件，再 mount 它。要支持这个场景，需要能把普通文件当成块设备用。

MossOS 为此实现了 LoopFileBlockDevice：

```
pub struct LoopFileBlockDevice {
    file: FileBackend,    // 被当作块设备使用的普通文件
```

```
    blocks: u64,          // 文件总块数 = file.len() / block_size
}

impl BlockDriverOps for LoopFileBlockDevice {
    fn read_block(&self, block_id: u64, buf: &mut [u8]) -> DevResult {
        let offset = self.checked_range(block_id, buf.len())?;
        // 循环 read_at 直到读完，处理短读
    }
    fn write_block(&self, block_id: u64, buf: &[u8]) -> DevResult {
        // 块号 → 文件偏移的换算：offset = block_id * block_size
    }
    fn flush(&self) -> DevResult {
        self.file.sync(false)  // 将脏数据写回底层文件
    }
}
```

代码 5-2　LoopFileBlockDevice 将普通文件适配为块设备接口

核心逻辑是块号到文件偏移的换算：块设备第 N 块对应普通文件从 `N * block_size` 开始的一段内容。ext4 后端仍按块读写，`LoopFileBlockDevice` 把这些块访问翻译成对镜像文件的 `read_at/write_at`。`sys_mount` 在识别到 `/dev/loop*` 路径时，将对应文件包装后传给 ext4 后端，镜像文件挂载走普通 ext4 路径，不需要单独处理。读写循环处理了短读/短写（底层文件系统可能一次未返回全部请求字节），只读 loop 设备以非 `MS_RDONLY` 挂载时显式拒绝。

5.3　inode 元数据缓存

频繁的 `stat` 调用和读写伴随的时间戳更新如果每次都穿透到底层文件系统，会在 I/O 密集场景产生大量元数据写——进程只是想读文件，却要附带一次写的开销。StarryOS 的 `metadata()` 直接访问底层节点，未引入额外的缓存层。MossOS 在 VFS 层增加了一套 inode 元数据缓存，以减少频繁穿透底层文件系统带来的开销。

缓存以 `(fs_ptr, inode)` 为键，同一文件被多个进程、多个 fd 或多条路径访问时共享同一份元数据。全局索引划分为 64 个分片，每个分片使用独立的 `RwLock`；冷加载和写回分别使用可睡眠锁，避免慢速文件系统访问占用自旋锁。目录同样进入缓存，目录项增删后主动使相关缓存失效；`NON_CACHEABLE` 节点则始终直接读取底层信息。

元数据更新先合并到缓存，再在 `fsync`、全局同步、缓存失效或卸载时提交。atime 默认遵循 relatime：只有 atime 不晚于 mtime 或 ctime，或者距上次更新超过 24 小时时才记录新值；`MS_NOATIME` 和 `MS_STRICTATIME` 可覆盖该策略。这样既能合并重复更新，也避免普通文件关闭时产生不必要的同步写入。

5.4　Folio 文件页缓存

MossOS 使用 Folio 作为普通文件的数据缓存单元。一个 Folio 由 1 到 16 个连续的 4 KiB 页面组成，并统一记录加载、脏页、写回和截断状态。相邻页面可以成组加载和写回，随机访问仍可退化为单页处理。

同一 inode 的不同 fd 和进程共享一份文件地址空间。文件缓存和 Folio 索引均采用 64 个分片，缓存容量不再按单个文件固定为 256 页，而是根据可用内存水位回收较冷的 Folio。Direct I/O 读前会写回重叠范围内的脏缓存，写后使对应的干净缓存失效，从而保持两条 I/O 路径的一致性。

`readahead` 和 `POSIX_FADV_WILLNEED` 会把预读任务交给后台工作队列，单次最多处理 1 MiB；文件映射缺页也会按访问模式预取相邻页面。Folio 在 BuildStorm 场景中的锁粒度、并发缺页和性能优化已在专项报告中展开，本节只说明其在文件系统中的基础作用。

5.5 扩展属性

扩展属性（xattr）是附加在文件上的键值对，可用于 ACL、安全标签和文件校验信息等场景。StarryOS 尚未包含相关接口，MossOS 补充了系统调用和 VFS 接口，并实现了当前所需的内存存储路径。

层次	MossOS 实现
syscall 层	12 个系统调用：setxattr/getxattr/listxattr/removexattr × 三个变体（路径版、l-符号链接版、fd 版）
VFS trait 层	NodeOps 新增 4 个方法（set_xattr/get_xattr/list_xattr/remove_xattr），默认返回 OperationNotSupported；Location 通过 inherit_methods 透出对应方法
存储层	`XattrStore`（`Mutex<BTreeMap<Vec<u8>, Vec<u8>>>`）挂在目录项关联的 TypeMap 上并按需创建；当前系统调用使用该内存存储

表 5-3　xattr 三层实现

实现中处理了几个规范语义：

- 命名空间：支持 `user.`、`security.`、`trusted.`、`system.` 四个前缀，`user.` 仅允许普通文件和目录使用；
- 标志语义：`XATTR_CREATE` 要求属性不存在，`XATTR_REPLACE` 要求属性已经存在；
- 大小限制：属性名最长 255 字节，值最长 65536 字节；
- **inode flags** 集成：设置属性时，immutable 或 append-only 文件会被拒绝。

ext4 后端已经提供直接读写磁盘 xattr 的 VFS 方法，但尚未接入上述系统调用路径，因此当前扩展属性不会持久化到 ext4 磁盘。

5.6 文件事件通知

用户态程序需要知道文件系统上发生了什么——备份工具感知文件修改，安全软件在文件访问时介入。Linux 提供 inotify 和 fanotify 两套机制覆盖不同场景。StarryOS 尚未包含这两套接口，MossOS 补充了主要路径。

5.6.1 inotify

inotify 面向具体文件或目录的变化监视。用户态对目标路径添加 watch，之后 create、delete、modify、attrib、move 等操作产生的事件进入事件队列，通过 `read` 取出。

实现上，一个 inotify fd 维护两张表：

```
struct Target { dev: u64, ino: u64, is_dir: bool }
struct Watch  { wd: c_int, target: Target, mask: u32 }
struct Event  { wd: c_int, mask: u32, cookie: u32, name: Option<String> }

pub struct InotifyFd {
    watches: Mutex<Vec<Watch>>,          // 当前 fd 上注册的所有 watch
    events: Mutex<VecDeque<Event>>,      // 等待用户态 read 的事件队列
```

```
    poll: PollSet,                          // 新事件到达时唤醒 read/poll
}
```

代码 5-3　inotify 的 watch 表与事件队列

事件投递发生在 VFS 层的创建、删除、重命名、属性更新路径上——这些路径在 StarryOS 中已经存在，MossOS 的工作是在这些路径上插入通知点。移动操作产生成对的 `IN_MOVED_FROM` 和 `IN_MOVED_TO` 事件，通过相同的 cookie 关联，让接收方能识别是同一次 rename。卸载时向所有相关 watch 发送 `IN_UNMOUNT`。

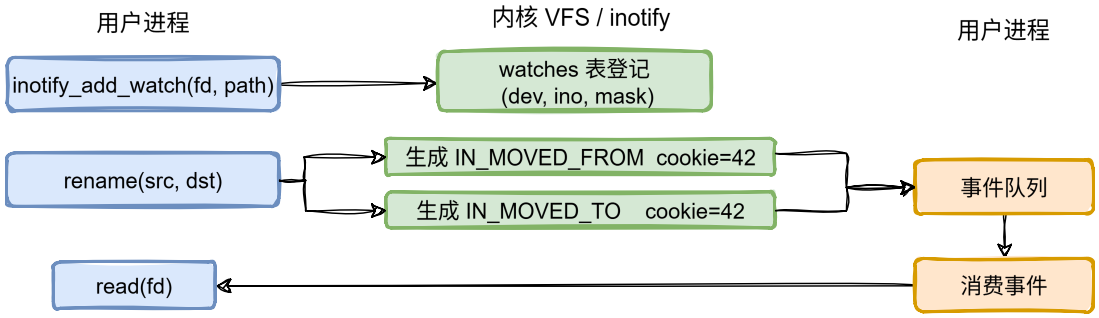


图 5-3　inotify 从注册 watch 到用户态取出事件的完整数据流

5.6.2　fanotify

fanotify 解决 inotify 覆盖不了的两个场景：监视范围可以是一整个挂载点甚至文件系统（而非单个路径），以及能在访问发生时主动介入（而非事后通知）。

mark 可以挂在单个文件、目录或整个挂载点上：

```
enum FanotifyMarkKind { Inode, Mount, Filesystem }

struct FanotifyMark {
    target: FanotifyTarget,
    kind: FanotifyMarkKind,     // mark 的作用范围
    mask: u64,                  // FAN_OPEN/FAN_ACCESS/FAN_CREATE 等
    ignored_mask: u64,          // 临时忽略的事件集合
}

struct FanotifyEvent {
    mask: u64,
    file: Option<Arc<dyn FileLike>>,  // 需要返回 fd 时保留文件对象
    fid: Option<FanotifyTarget>,      // FID 模式下报告的文件身份
    permission_id: Option<u64>,       // 权限事件等待用户态响应
}

pub struct FanotifyFd {
    marks: Mutex<Vec<FanotifyMark>>,
    events: Mutex<VecDeque<FanotifyEvent>>,
    permissions: Mutex<Vec<FanotifyPermission>>,  // 尚未 allow/deny 的权限事件
    poll: PollSet,
}
```

代码 5-4　fanotify 的 mark、事件队列与权限等待表

权限事件（FAN_OPEN_PERM、FAN_ACCESS_PERM）是 fanotify 最特殊的部分：内核在文件被打开或访问时先等待监听进程响应，再决定是否放行。触发进程在此期间阻塞，监听进程通过写入 allow 或 deny 响应。安全软件（防病毒、DLP）依赖这个机制在文件访问路径上做拦截。

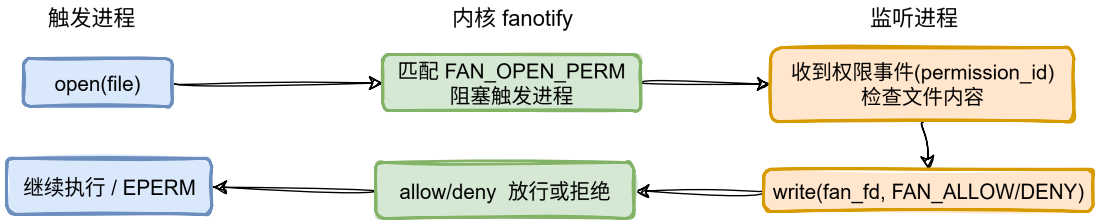


图 5-4　fanotify 权限事件的三方交互时序

5.6.3 无监听快速路径

inotify 和 fanotify 需要在 read、write、open、close 等热路径上插入事件投递逻辑。MossOS 用全局计数和掩码提前过滤：inotify 的全局 watch 计数为零时，通知函数通过一次原子读取直接返回；fanotify 先检查全局兴趣掩码，当前事件无命中时跳过后续处理，命中后再检查仍然有效的 fd 和 mark。这样可以降低未监听事件在普通 I/O 路径上的额外开销。

5.7 文件 I/O 测试

初赛阶段曾使用 iozone 在 QEMU RISC-V 环境中对文件 I/O 吞吐量进行测试。下表保留当时记录的优化前基线、MossOS 测量值和 Ubuntu 参考值，用于说明初赛阶段文件缓存改造的效果；决赛阶段针对 BuildStorm 继续完成的文件系统性能优化及测试数据见专项报告。

测试场景	优化前	初赛 MossOS	Ubuntu 参考
顺序写	3524	33221	51922
顺序读	13136	81470	104754
顺序重写	7471	81639	72760
随机读	11082	57668	57175
随机写	7141	59978	59817
反向读	10743	58824	67361
pread 并发读	14184	108348	125270
pwrite 并发写	3724	28263	39335

表 5-4　初赛阶段 iozone 性能记录（glibc，单位 kB/sec）

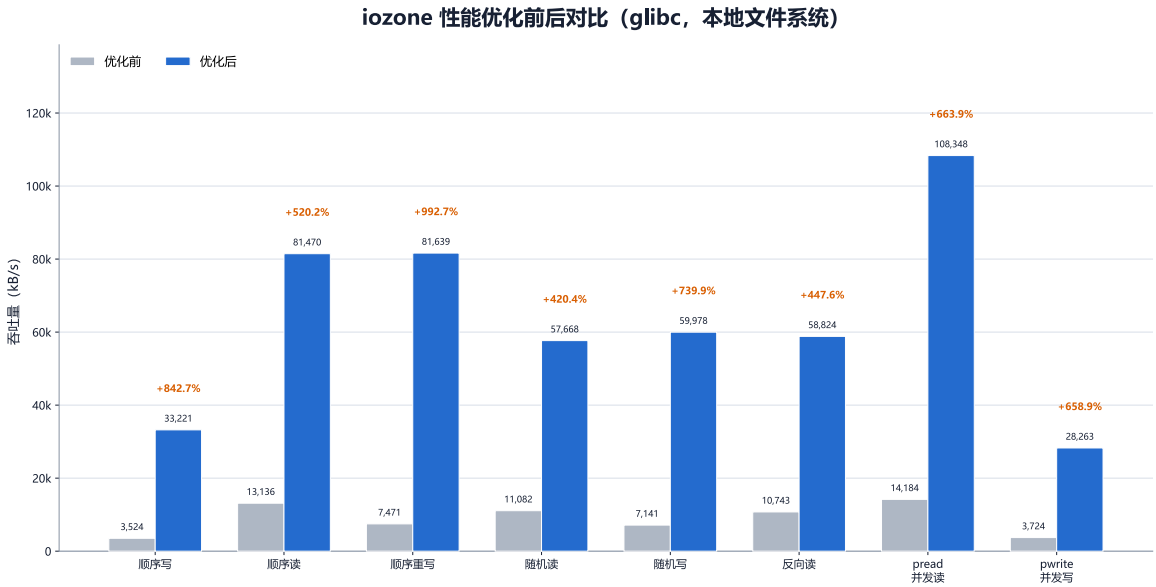


图 5-5　初赛阶段 iozone 吞吐量对比（glibc）

测试场景	优化前	初赛 MossOS	Ubuntu 参考
顺序写	3524	26407	44818
顺序读	13136	62733	98518
顺序重写	7471	62254	69316
随机读	11082	43813	91905
随机写	7141	51344	57648
反向读	10743	54217	78861
pread 并发读	14184	88315	141988
pwrite 并发写	3724	28192	49303

表 5-5　初赛阶段 iozone 性能记录（musl，单位 kB/sec）

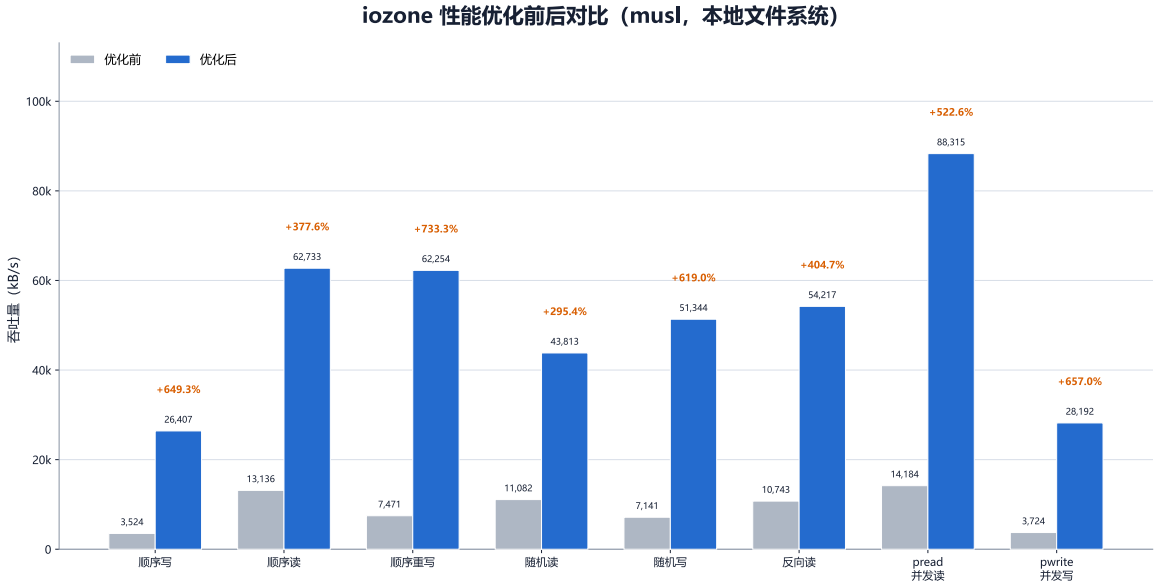


图 5-6　初赛阶段 iozone 吞吐量对比（musl）

两组 libc 的测试项均高于优化前基线，其中 glibc 环境下的顺序重写、随机读写已达到同期 Ubuntu 参考值附近。由于这些数据采集于初赛阶段，不作为当前决赛版本的最终性能结论。

5.8 本章小结

本章介绍了 MossOS 对文件系统功能和缓存结构的完善。挂载子系统补充了 bind、move、remount、卸载检查、新式挂载 API 和 Loop 设备；inode 元数据缓存通过分片索引、relatime 和延迟提交减少底层访问；Folio 为普通文件 I/O、预读和文件映射提供统一的数据缓存；扩展属性、inotify 和 fanotify 则补齐了相关 Linux 接口。文件系统支持范围和扩展属性持久化仍有明确边界，后续工作将在现有 VFS 接口上继续完善。

第 6 章 信号机制

信号是进程间传递异步事件的基本手段。进程之间可以用信号通知对方，内核也借助信号把硬件异常、终端中断、子进程状态变化等事件交给进程处理。

MossOS 沿用 StarryOS 的信号框架，重点补充了停止与继续状态、父进程对子进程状态变化的感知，以及 `kill` 的权限校验。

6.1 StarryOS 的信号机制

6.1.1 整体结构

StarryOS 的信号机制由底层库 `starry-signal` 提供核心语义，内核负责将其接入任务管理、系统调用以及返回用户态的处理路径。信号管理器分两层组织：

- 进程级：保存共享的待处理信号队列、信号处理动作表，以及进程内所有线程的引用。进程级信号只入队一次，内核唤醒未屏蔽该信号的线程，最终由其中一个线程取出并处理。
- 线程级：保存线程私有的待处理信号、屏蔽集和信号栈。

标准信号只保留一份，重复发送会合并；实时信号按发送顺序排队，每次发送都不丢。

6.1.2 信号检查与递送

每次从内核返回用户态之前都会检查信号。内核从队列里取出一个未屏蔽的信号，根据处理动作决定后续行为——终止、忽略、停止，或者跳到用户注册的处理函数。

调用用户处理函数时，内核在用户栈上构造信号栈帧，保存被中断时的寄存器状态和恢复所需的上下文，再把控制流转过去。函数执行完后，通过 `rt_sigreturn` 回到内核，恢复栈帧并继续原来的执行流。

`kill`、`tkill`、`tgkill`、`sigaction`、`sigprocmask`、`sigsuspend`、`sigtimedwait` 等系统调用都建立在这条链路上，MossOS 沿用了这些系统调用的实现。

6.1.3 现存问题

StarryOS 的实现存在三个明显缺口：

1. 停止与继续没有实现。收到 `SIGSTOP`、`SIGTSTP` 时直接让进程退出，对 `SIGCONT` 不做处理。Shell 的作业控制因此完全无法工作。
2. **`kill`** 没有权限校验。普通用户理论上只能向凭据匹配的进程发信号，这个检查在 StarryOS 里缺失。
3. 父进程看不到子进程的停止与继续。`waitid` 的 `WSTOPPED`、`WCONTINUED` 标记形同虚设。

后面几节分别说明 MossOS 如何处理这三个问题。

6.2 进程可暂停状态

6.2.1 问题所在

StarryOS 里进程只有两条路：继续运行，或者退出。`SIGSTOP` 走的是退出路径，但 POSIX 标准规定停止信号应该让进程暂停、保留现场，等 `SIGCONT` 到来后再从原位置继续——停止不等于死。

Shell 按下 Ctrl+Z 时最能说明这个问题。前台进程收到 `SIGTSTP` 后应暂停，用户执行 `fg` 并发送 `SIGCONT` 后再从原位置继续。StarryOS 原有实现会直接结束进程，无法表达这种状态变化。

6.2.2 核心结构

要支持暂停，进程需要一个对外可见的“暂停中”状态，以及一个让线程真正挂住自己的地方。MossOS 引入 `ProcessWaitStatus` 枚举，并在 `ProcessData` 上新增两个字段：

```rust
pub enum ProcessWaitStatus {
    Stopped(Signo),   // 被停止信号挂起，记录触发的信号编号
    Continued(Signo), // 被继续信号唤醒，记录触发的信号编号
}

pub struct ProcessData {
    // ...
    // 父进程通过 waitid 可见的暂停/继续状态
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // 被停止的线程在此事件上挂起，等待继续或退出路径唤醒
    pub stop_event: Arc<PollSet>,
}
```

代码 6-1　进程可暂停状态的核心结构

`wait_status` 保存父进程可观察的停止或继续事件，`waitpid` 和 `waitid` 都会读取它；`stop_event` 则用于挂起实际处理停止信号的线程。

6.2.3 处理路径

信号检查路径里，停止和继续两条分支现在变成了：

```rust
SignalOSAction::Stop => {
    // 写入对外可见的停止状态
    *thr.proc_data.wait_status.lock() =
        Some(ProcessWaitStatus::Stopped(signo));
    // 通知父进程可以从 waitid 取结果了
    wake_waiting_parent(&thr.proc_data);
    // 线程在 stop_event 上挂起，等待被唤醒
    wait_for_continue(thr);
}
SignalOSAction::Continue => {
    *thr.proc_data.wait_status.lock() =
        Some(ProcessWaitStatus::Continued(signo));
    wake_waiting_parent(&thr.proc_data);
    // stop_event 上的线程在信号发送路径里被唤醒
}
```

代码 6-2　停止与继续信号的处理路径

停止路径依次记录状态、通知父进程，并让当前线程等待 `stop_event`。发送 `SIGCONT` 时，发送路径会立即写入继续状态并唤醒等待线程；`SIGKILL` 也会唤醒它，使线程能够进入退出流程。当前实现只挂起实际处理停止信号的线程，尚未实现多线程进程中所有线程同步停止的组停止语义。

图 6-1 展示了作业控制希望表达的状态关系：停止与继续在运行和暂停之间切换，退出后进入僵尸状态，父进程调用 `wait` 完成最终回收。当前代码通过 `wait_status` 和线程阻塞近似表达暂停状态，并未在调度器中增加独立的进程停止状态。

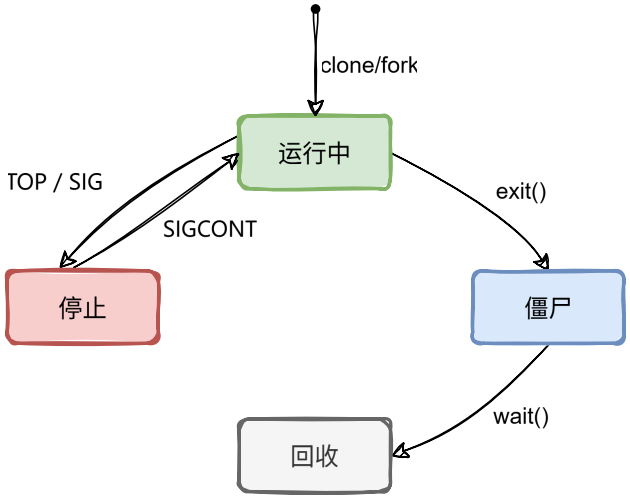


图 6-1　作业控制的目标状态关系

6.3 信号驱动的 wait 语义

停止和继续是由信号引起的状态变化，父进程能否观察到它们取决于 `wait` 系列调用的选项。`waitpid` 不带选项时只报告退出；`waitid` 则要求调用者至少指定 `WEXITED`、`WSTOPPED` 或 `WCONTINUED` 中的一项。

选项	作用
`WUNTRACED` / `WSTOPPED`	`waitpid` 使用 `WUNTRACED`，`waitid` 使用同值的 `WSTOPPED`，用于报告子进程停止状态
`WCONTINUED`	允许报告子进程继续运行状态
`WNOHANG`	当前没有可报告状态时立即返回，不阻塞父进程
`WNOWAIT`	`waitid` 读取状态但不消费，后续仍可再次查询

表 6-1　子进程等待的主要选项

以 Shell 为例。Ctrl+Z 向前台进程组发送 `SIGTSTP`，Shell 需要区分子进程是收到停止信号还是已经退出；`WUNTRACED` 用于读取停止事件，`WCONTINUED` 用于读取后续的继续事件。MossOS 已能传递这些状态通知，但多线程进程的完整组停止仍未实现。

如 图 6-2 所示，停止、继续和退出都可以成为一次 `wait` 的报告，只有退出事件会进入僵尸回收流程。当前代码消费停止事件时还会清除 `wait_status` 并唤醒停止线程，因此它实现的是基本兼容路径，尚未达到 Linux 中“读取停止通知不改变停止状态”的完整语义；`waitid` 配合 `WNOWAIT` 时则只读取而不消费该状态。

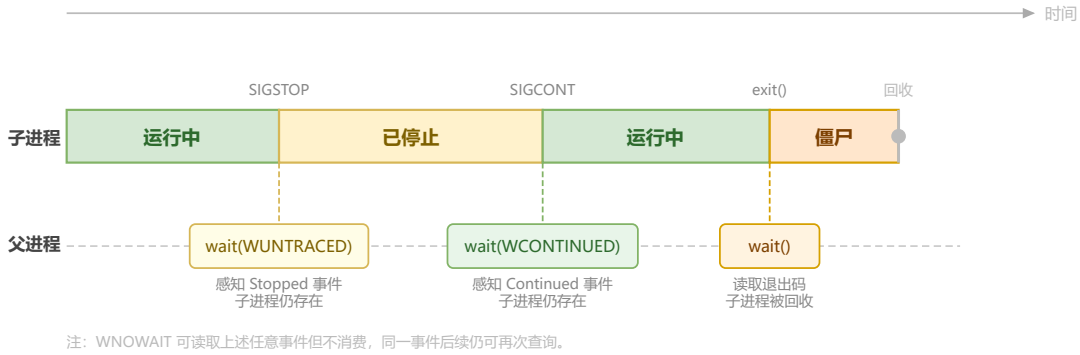


图 6-2　子进程停止、继续、退出三类事件与父进程 wait 消费的时序

`waitid` 相比 `waitpid` 的好处是返回结构化的 `siginfo_t`，能直接区分正常退出、信号终止、停止和继续，不需要调用者手动解析整数状态。`waitid` 还支持通过 pidfd 选择目标——pidfd 是指向进程对象的文件描述符，不会随 PID 复用而改变指向，配合 `WNOWAIT` 可以先查看状态、之后再完成回收。父进程没有可报告状态时如何进入阻塞、状态变化后怎样被唤醒，见下一节。

6.4　父进程如何感知状态变化

6.4.1　通知路径

子进程状态变化发生在子进程自己的执行流里，父进程可能正阻塞在 `waitid` 上。这里需要一套机制：状态变化时，主动唤醒正在等待的父进程。

StarryOS 已有一个 `child_exit_event`，但只在子进程退出时触发。停止和继续没有对应的唤醒动作，所以 `WSTOPPED`、`WCONTINUED` 的语义一直没法生效。

MossOS 没有新建通道，而是让停止和继续两条路径也触发同一个 `child_exit_event`，把它的语义扩展为“子进程任何状态变化都触发”：

```
fn wake_waiting_parent(proc_data: &ProcessData) {
    if let Some(parent) = proc_data.proc.parent()
        && let Ok(parent_data) = get_process_data(parent.pid())
    {
        parent_data.child_exit_event.wake();
    }
}
```

代码 6-3　子进程状态变化时唤醒父进程

停止路径和继续路径在写完 `wait_status` 之后都调用这个函数（见 代码 6-2）。父进程被唤醒后重新读 `wait_status`，就能拿到子进程的最新状态。

6.4.2　防止通知丢失

父进程登记唤醒入口（Waker）和子进程状态变化之间有一个竞争窗口：父进程检查完 `wait_status`、还没把 Waker 挂上去，子进程的状态恰好先变了，这次通知就悄悄丢了，父进程会一直等下去。

MossOS 用登记后复查覆盖这个窗口：

```
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
```

```
            // 暂无结果，登记父进程的 Waker
            child_exit_event.register(cx.waker());
            // 登记完立刻再查一次，覆盖登记窗口里发生的状态变化
            match check_children().transpose() {
                Some(result) => Poll::Ready(result),
                None => Poll::Pending,
            }
        }
    }
```

代码 6-4　父进程等待子进程状态，登记后复查防止通知丢失

第一次检查没结果，把 Waker 登记上去，然后立刻再查一次。如果第二次查到了结果，说明状态变化恰好落在那个窗口里，直接返回不需要等待。如果仍然没有，才放心地进入阻塞。这套双重检查的通用模式在第三章阻塞与唤醒机制里有更完整的说明。

6.5　`kill` 的信号权限校验

POSIX 规定，除 root 外，发送者只能向与自己 UID 或 SUID 匹配的目标发信号。`SIGCONT` 有个例外：同一会话内可以跨 UID 发，让 Shell 能对已切换身份的子进程发出恢复命令。

StarryOS 的 `ProcessData` 里没有 UID 相关字段，无法完成这项检查。MossOS 在进程与线程管理一章引入 real/effective/saved 三元凭据，`kill` 使用这些字段判断发送权限：

条件	允许发送
发送者有效 UID 为 0（root）	是
发送者 UID 或 EUID 与目标 UID 或 SUID 匹配	是
发送 `SIGCONT` 且与目标处于同一会话	是
其余情形	否

表 6-2　kill 的信号发送权限规则

`sys_kill` 支持指定进程、当前进程组、广播和指定进程组四种目标语义。批量发送时，至少有一个目标通过权限检查即返回成功；存在目标但均无权限时返回 `EPERM`，没有匹配目标时返回 `ESRCH`。这套凭据检查目前只接入 `kill`，`tkill`、`tgkill`、实时信号排队接口和 `pidfd_send_signal` 尚未统一使用该检查。

6.6　小结

StarryOS 已经完成了信号发送、入队、屏蔽与用户处理函数递送的基本链路。MossOS 在此基础上补充了以下环节：

补充内容	核心结构	支撑的使用场景
停止与继续状态	`ProcessWaitStatus`、`stop_event`	单线程任务的基本暂停与恢复
信号驱动 wait 语义	`WUNTRACED` / `WSTOPPED`、`WCONTINUED`	父进程感知子进程停止与继续事件
状态变化通知	`child_exit_event` 复用，登记后复查	`waitid` 停止与继续事件正常到达父进程
`kill` 权限	UID/EUID/SUID 三元凭据校验	`kill` 的跨 UID 访问控制

表 6-3　MossOS 对信号机制的补充

当前实现仍有两项边界：停止信号尚未让多线程进程中的所有线程同步停止，消费停止事件也会唤醒对应线程；信号权限检查目前只覆盖 `kill`。这些部分仍需在后续实现中继续完善。

第 7 章 网络模块

网络模块负责组织网络数据的收发与传输。MossOS 基于 StarryOS 改造而来，网络栈的整体框架沿用了 StarryOS 已有的设计。StarryOS 基于 smoltcp 协议栈实现了 TCP/UDP socket 的基本功能，MossOS 在此基础上针对若干性能瓶颈与兼容性缺口进行了补充。本章先介绍 StarryOS 网络栈的整体结构与现存问题，随后说明 MossOS 所做的增量。

7.1 StarryOS 的网络栈

7.1.1 整体结构

StarryOS 基于开源协议栈 smoltcp 构建网络模块。smoltcp 面向嵌入式和 `no_std` 环境，协议实现轻量。StarryOS 通过 `axnet` 模块对 smoltcp 进行封装，向上层提供 `socket`、`bind`、`connect`、`send`、`recv` 等接口。设备层支持以太网和 loopback 设备，MossOS 在 `axnet` 中使用动态分配的 socket buffer 和报文队列。

应用程序通过 socket 系统调用收发数据，外部流量经协议栈处理后由网络设备收发，设备事件和 socket 操作共同推进协议栈状态。StarryOS 已完成基本收发框架，但在地址族支持、buffer 配置和 loopback 路径等方面仍有改进空间。

7.1.2 现存问题

StarryOS 的网络栈在简单 TCP/UDP 通信场景下可以工作，但在复杂负载下暴露出若干问题：

1. 地址族支持不完整。StarryOS 虽然接收 `AF_INET` 和 `AF_INET6` 参数，但内部统一按 IPv4 处理。IPv4-mapped IPv6 地址（如 `::ffff:127.0.0.1`）会导致路由查找失败。iperf 等工具在 glibc/musl 环境下会混用 IPv4、IPv6 和 mapped 地址，基线无法正确处理这些组合。
2. **Socket** 选项语义缺失。缺少 iperf/netperf 等性能测试工具依赖的关键选项。`TCP_INFO` 返回固定假值，`SO_SNDBUF`/`SO_RCVBUF` 不按 Linux 语义翻倍，`IP_RECVERR`、`SO_DONTROUTE` 等返回 `ENOPROTOOPT` 导致应用进入异常路径。
3. **Buffer** 容量偏小。TCP 和 UDP 的发送/接收 buffer 均为 64KB，在高吞吐场景下成为瓶颈。netperf 的 TCP_STREAM 测试仅能达到约 11 Mbits/sec，远低于预期。
4. **Loopback** 路径未优化。本机 `127.0.0.1` 流量仍走完整的外部网络路径，经过多次拷贝和排队。每次 `send()`/`recv()` 都无条件推进整个网络栈，累积的轮询开销显著。

7.2 地址族支持与兼容性补齐

7.2.1 IPv4/IPv6 混用的处理

IPv4 和 IPv6 在 Linux 中并不是完全独立的两套协议。Linux 允许 IPv6 socket 接收 IPv4 连接（通过 IPv4-mapped IPv6 地址），也允许通过 `IPV6_V6ONLY` 选项禁用这一行为。StarryOS 对地址族的处理过于简化，`sys_socket()` 虽然接收不同的 `domain` 参数，但创建的 socket 都按 IPv4 处理。

问题的关键在于地址格式无法匹配。iperf 测试 `::1`（IPv6 loopback）时会创建 IPv6 socket 并传入 IPv6 地址，但 StarryOS 内部按 IPv4 处理，路由查找时发现地址类型不匹配而失败。更复杂的情况是 IPv4-mapped IPv6 地址（`::ffff:127.0.0.1`），它在形式上是 IPv6，但语义上表示 IPv4，需要正确转换才能路由。

MossOS 为 TCP 和 UDP socket 保存显式的 `IpFamily` 标识，并将地址族检查贯穿 bind、connect 和 listen 匹配过程。

创建 socket 时根据 `AF_INET` 或 `AF_INET6` 保存对应的 family。Bind 和 connect 操作在接收地址参数时，先检查地址类型是否与 socket family 匹配。对于 IPv6 socket，若地址是 IPv4-mapped 形式且 `ipv6_only` 为 false，则提取其中的 IPv4 部分进行路由；否则按 IPv6 处理。

Listen table 在接收连接时也需配合检查。IPv6 wildcard listener（`::`）在 `ipv6_only` 为 false 时可以接收 IPv4 连接（以 mapped 形式呈现），但在 `ipv6_only` 为 true 时只能接收纯 IPv6 连接。MossOS 在 listen table 的匹配逻辑中增加了 family 和 `ipv6_only` 的判断，确保连接被正确路由到对应的 listener。

当前物理网卡仍配置 IPv4 地址和默认路由；IPv6 支持主要覆盖 socket 地址族语义、IPv4-mapped IPv6 和本机 `::1` 通信，尚不代表完整的外部 IPv6 网络能力。

7.2.2 Socket 选项的补齐

iperf 和 netperf 等性能测试工具在启动时会设置若干 socket 选项以调整行为。StarryOS 对部分选项的处理不符合 Linux 语义，导致工具进入异常路径或直接失败。MossOS 针对若干关键选项进行了补齐：

选项	MossOS 的处理
`TCP_INFO`	返回最小兼容结构，提供连接状态、MSS 和 PMTU；RTT、重传等统计尚未完整实现
`SO_SNDBUF` / `SO_RCVBUF`	按 Linux 语义，用户请求的值在内核中翻倍存储；读取时返回实际分配的值
`TCP_QUICKACK`	在 smoltcp 路径上调整 ACK 延迟；本地直达连接不经过 TCP 协议处理
`SO_LINGER`	支持超时为零时的中止式关闭；非零超时的完整等待语义尚未实现
`IP_RECVERR`	保存选项状态，尚未实现 ICMP 错误队列
`SO_DONTROUTE`	保存选项状态，尚未改变实际路由行为

表 7-1　补齐的 Socket 选项

这些接口避免了应用因选项缺失而直接失败。其中部分选项已实现实际行为，部分目前仅完成兼容性状态维护。

7.3 Buffer 扩展

StarryOS 原有的 64KB buffer 在高吞吐场景下容易成为瓶颈。发送方或接收方频繁因 buffer 满而阻塞，限制了吞吐上限。MossOS 将 buffer 容量提升至适合高吞吐场景的配置：

```
// TCP buffer 从 64KB 提升到 512KB
const DEFAULT_TCP_RX_BUF_LEN: usize = 512 * 1024;
const DEFAULT_TCP_TX_BUF_LEN: usize = 512 * 1024;

// UDP buffer 从 64KB 提升到 1MB
const DEFAULT_UDP_RX_BUF_LEN: usize = 1024 * 1024;
const DEFAULT_UDP_TX_BUF_LEN: usize = 1024 * 1024;

// Router packet buffer 提升到 256 项
const ROUTER_PACKET_BUFFER_SIZE: usize = 256;
```

代码 7-1　Buffer 容量配置

扩展 buffer 降低了高吞吐场景下因容量不足而阻塞的频率。

7.4 Loopback 路径优化

7.4.1 优化动机

本机流量和外部流量的优化目标完全不同。外部网络需要考虑链路 MTU、拥塞窗口、丢包重传；本机 loopback 是内存内投递，不存在物理链路错误，也不应被网络拥塞算法限制。

StarryOS 的 loopback 流量仍走完整的网络栈路径：send buffer → 协议栈封包 → router queue → loopback device buffer → router RX queue → 协议栈解包，涉及多次拷贝和排队。每次 `send()`/`recv()` 都会无条件推进整个网络栈，即使当前 socket 已经有数据可读写，仍然要遍历所有 socket 并轮询底层设备。对于 loopback 这种纯内存操作，轮询开销占据了大部分 CPU 时间。

7.4.2 TCP/UDP 本地直达

MossOS 在 `axnet` 中引入本地传输层，优先处理目标为 loopback 的 TCP 和 UDP 通信。TCP 建立本地连接时，为两端创建成对的 `LocalTcpStream`；发送方直接写入对端的流缓冲区，并唤醒等待接收的任务。UDP 则根据本地端点查找目标 socket，将数据报放入其接收队列。两条路径都不再经过 smoltcp 封包、路由和解包过程。

本地传输层保留了地址族、wildcard bind、`IPV6_V6ONLY`、阻塞和就绪事件等 socket 语义。非 loopback 流量仍经过正常协议栈。

7.4.3 Router 直投与大 MTU

MossOS 在 router 层识别 IPv4 `127.0.0.0/8` 和 IPv6 `::1` 地址。对仍进入协议栈的 loopback 报文，router 可将其直接投递到接收队列，跳过设备队列。

同时，loopback 设备使用独立的 MTU 配置（65535 字节），不受外部网卡 1500 字节限制。大 MTU 减少了分片和重组开销，对大数据块传输的性能提升显著。

7.4.4 TCP 拥塞窗口绕过

TCP 协议的拥塞窗口机制用于防止网络过载，但 loopback 流量不存在网络拥塞。StarryOS 的 smoltcp 对所有 TCP 连接统一应用拥塞控制，loopback 连接的发送速率被人为限制。

MossOS 在 smoltcp 层面识别 loopback tuple，绕过拥塞窗口检查，并调整 Nagle 算法和延迟 ACK 行为。这些优化作用于仍经过 smoltcp 的 loopback TCP 路径；命中本地传输层的 TCP 连接则直接在 socket 之间搬运数据。

7.4.5 按需轮询策略

StarryOS 在每次 `send()`/`recv()` 时都会调用网络栈的 poll 函数，遍历所有 socket 并推进协议栈状态。对于已经有数据的 loopback socket，这一轮询是不必要的。

MossOS 改为优先尝试收发操作，在遇到 `WouldBlock` 并进入等待时推进协议栈；连接建立或报文发送等需要立即推进状态的位置仍会主动 poll。本地直达路径主要通过缓冲区状态和等待队列直接唤醒对端。

`poll()`/`select()`/`epoll_wait()` 等 I/O 多路复用接口在检查 fd readiness 前统一推进一次网络栈，避免每个 socket 重复推进。

7.5 初赛阶段性能记录

初赛阶段曾使用 iperf3 和 netperf 在 QEMU RISC-V 环境中测试 loopback 性能。下表保留当时记录的优化前基线、MossOS 测量值和 Ubuntu 参考值，用于说明初赛阶段网络栈改造的效果。这些数据早于当前本地 TCP/UDP 传输层，不作为决赛版本的最终性能结论。

7.5.1 iperf3 测试结果

测试场景	优化前 (Mbits/s)	初赛 MossOS (Mbits/s)	Ubuntu 参考 (Mbits/s)
TCP 基本测试（glibc）	265	1608	3041
TCP 并发测试（glibc）	239	1935	3574
TCP 反向测试（glibc）	283	1618	3277
UDP 基本测试（glibc）	36.4	1167	2908
UDP 并发测试（glibc）	19.6	1260	3840
UDP 反向测试（glibc）	38.9	1280	2744

表 7-2　初赛阶段 iperf3 性能记录（glibc，loopback）

表 7-2 给出了 glibc 环境下各测试项的原始吞吐量。图 7-1 进一步用柱状图直接对照优化前后的测试结果，并在柱顶标出相对提升幅度。

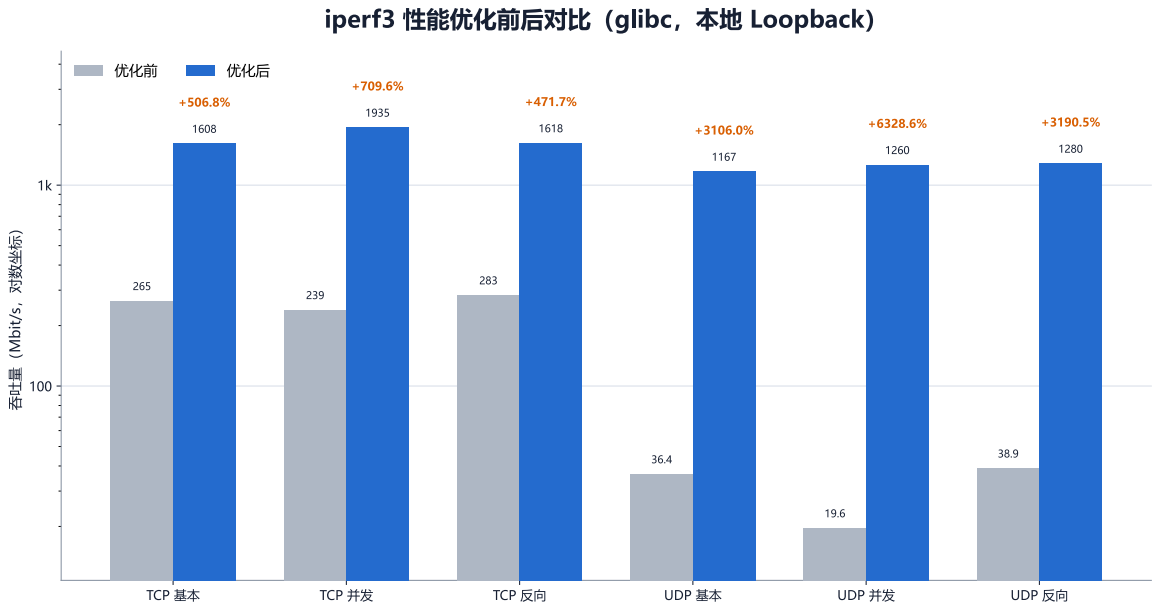


图 7-1　初赛阶段 iperf3 吞吐量对比（glibc）

musl 环境采用相同的测试场景和统计口径，原始吞吐量如下。

测试场景	优化前 (Mbits/s)	初赛 MossOS (Mbits/s)	Ubuntu 参考 (Mbits/s)
TCP 基本测试（musl）	245	1669	3267
TCP 并发测试（musl）	207	1997	2755
TCP 反向测试（musl）	278	1638	3379
UDP 基本测试（musl）	34.7	1178	2324
UDP 并发测试（musl）	23.9	1249	3809
UDP 反向测试（musl）	37.4	1270	2427

表 7-3　初赛阶段 iperf3 性能记录（musl，loopback）

图 7-2 使用相同方式展示 musl 结果。可以看到，TCP 和 UDP 各场景均有显著提升，其中 TCP 并发以及 UDP 基本、反向场景的改善尤为明显。

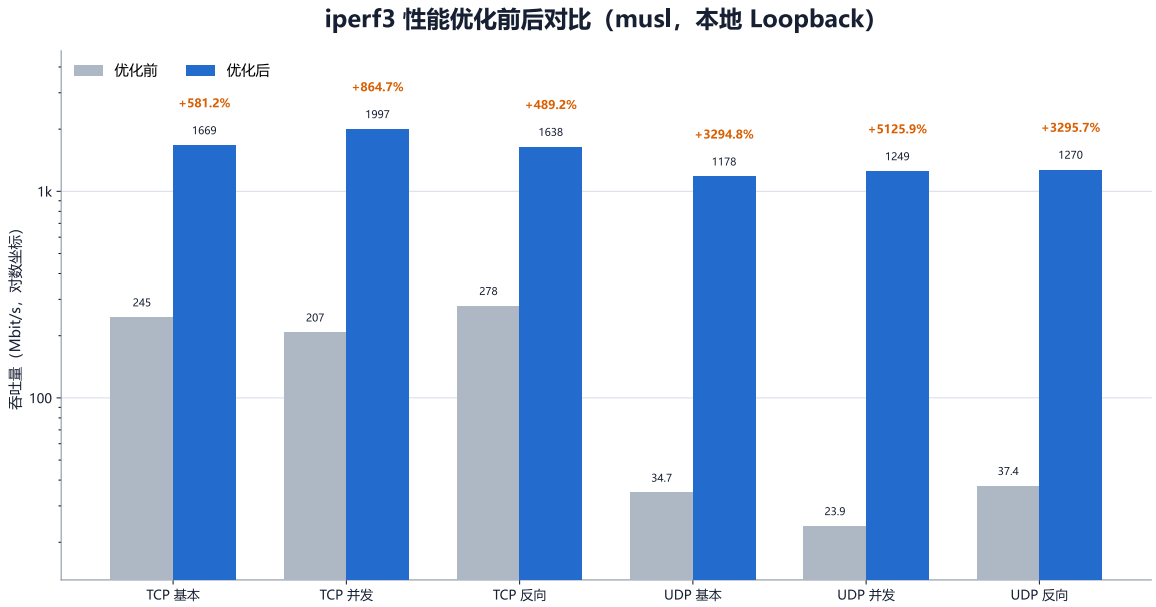


图 7-2　初赛阶段 iperf3 吞吐量对比（musl）

7.5.2　netperf 测试结果

iperf3 侧重持续吞吐，netperf 则进一步覆盖流式传输、请求响应和连接建立等不同负载。由于这些子项的原始单位并不完全相同，表格保留各自单位，后续柱状图也按单位分区展示。

测试场景	优化前	初赛 MossOS	Ubuntu 参考
TCP 流式传输 (Mbits/s)	11.01	299.35	277.37
UDP 流式传输 (传输率/秒)	159.85	192.11	127.63
TCP 请求响应 (传输率/秒)	1591.55	2681.80	2544.46
UDP 请求响应 (传输率/秒)	1644.41	2140.90	2807.66
TCP 连接请求响应 (传输率/秒)	146.30	274.08	753.23

表 7-4　初赛阶段 netperf 性能记录（glibc，loopback）

为避免把 Mbits/s 与传输率/秒混在同一坐标轴中，图 7-3 将 TCP_STREAM 单独绘制，其余事务率项目使用另一坐标区，并直接比较优化前后的原始结果。

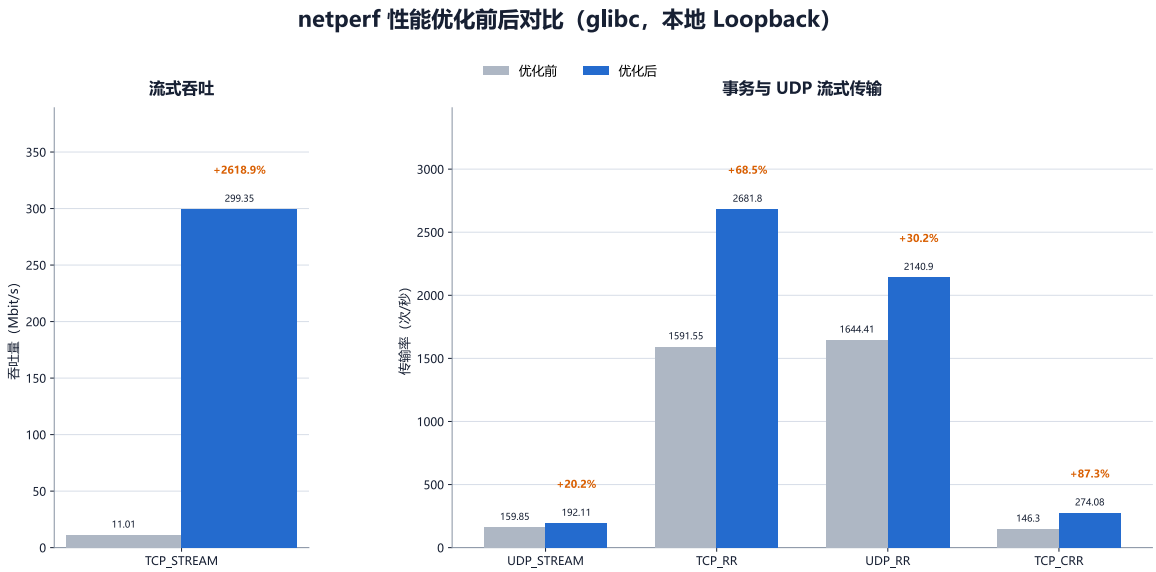


图 7-3　初赛阶段 netperf 性能对比（glibc）

初赛阶段记录中，TCP loopback 吞吐较优化前提升约 6–27 倍，UDP loopback 吞吐提升约 32–64 倍；netperf TCP_STREAM 从 11.01 Mbits/sec 提升至 299.35 Mbits/sec。这些结果只反映当时版本和测试环境。

7.6　小结

StarryOS 已经搭建起基于 smoltcp 的网络栈框架，提供了 TCP/UDP socket 的基本功能。MossOS 在此基础上针对若干性能瓶颈与兼容性缺口进行了补充：

方面	StarryOS	MossOS 增量
地址族	仅 IPv4	区分 IPv4/IPv6，支持 IPv4-mapped IPv6 和 `IPV6_V6ONLY`
Socket 选项	缺少关键选项	补齐 `TCP_INFO`、`SO_*BUF`、`TCP_QUICKACK` 等
Buffer 容量	TCP/UDP 均 64KB	TCP 512KB，UDP 1MB，Router 256 项
Loopback 优化	走完整网络栈	TCP/UDP 本地直达与事件唤醒，协议栈路径支持 Router 直投、大 MTU 和拥塞控制优化

表 7-5　MossOS 对网络模块的补充

这些增量使得 MossOS 能够运行 iperf、netperf 等网络性能测试工具。初赛阶段测试已显示 loopback 吞吐明显改善；当前本地 TCP/UDP 快速路径只对 loopback 地址启用，非 loopback 流量仍经过正常协议栈。

第 8 章 多架构硬件平台适配与设备驱动

8.1 平台适配概述

8.1.1 基于 ArceOS 的硬件抽象体系

MossOS 基于 ArceOS 组件化框架并从 StarryOS 演进而来。系统复用了 ArceOS 的 `axcpu`、`axplat`、`axhal` 和 `axdriver` 等基础组件，并针对决赛平台补充了新的平台 crate 和原生设备驱动。

这套硬件抽象按职责分为三层。`axcpu` 处理指令集相关的 Trap、寄存器和上下文切换；`axplat` 定义启动、内存、中断、时钟、串口和电源等平台接口；`axhal` 将这些接口包装为内核其余模块可直接调用的统一入口。具体平台通过 `#[impl_plat_interface]` 在编译期完成静态绑定，不需要在运行时通过大量条件分支选择实现。

如 图 8-1 所示，平台层和驱动层共同吸收机器差异，`axcpu` 则收敛指令集相关实现。

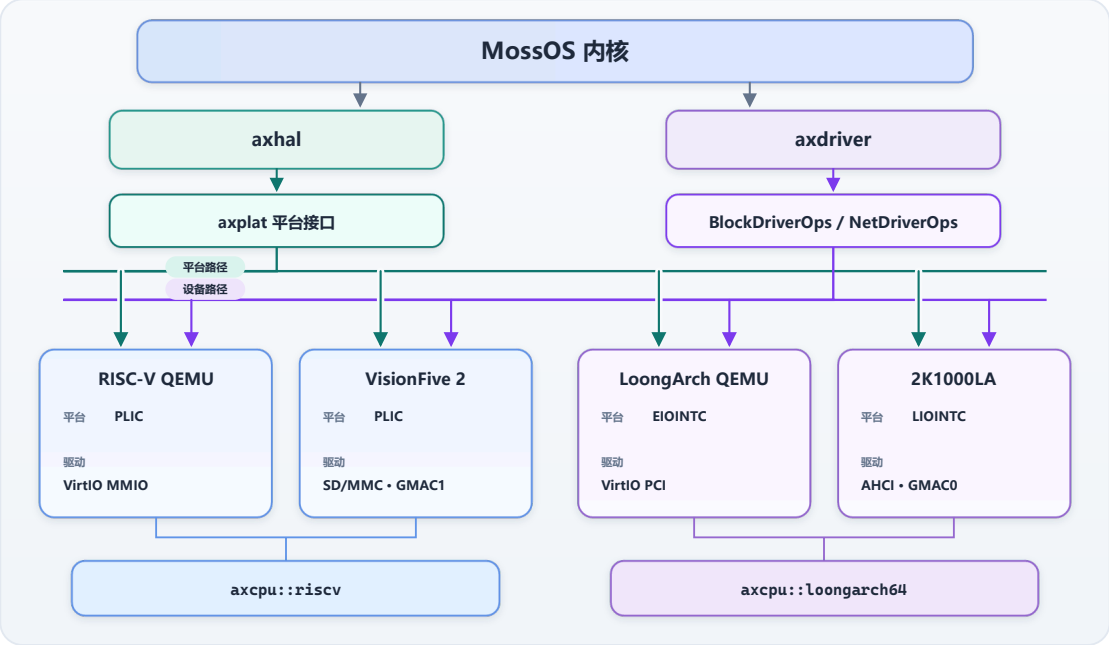


图 8-1　MossOS 多架构硬件抽象结构

8.1.2 从双架构 QEMU 到四平台支持

初赛阶段，MossOS 已能在 RISC-V64 QEMU virt 和 LoongArch QEMU virt 上运行，存储与网络主要依赖 QEMU 提供的 VirtIO 设备。决赛阶段又增加了 StarFive VisionFive 2 和 Loongson 2K1000LA 两个物理平台。真实硬件不再具有统一的 VirtIO 拓扑，内核需要接管固件传入的设备树，处理板级 CPU 编号、中断路由、存储控制器、网卡以及 DMA 一致性。

四个目标各由独立的平台配置和平台 crate 提供底层实现，上层调度、内存、文件系统和网络代码仍保持共享。

8.1.3 四个平台的软硬件结构

平台	启动与 CPU	中断机制	主要设备
RISC-V QEMU virt	OpenSBI，核数可配置	PLIC、SBI timer/IPI	VirtIO MMIO 块设备与网卡
LoongArch QEMU virt	QEMU 固件，核数可配置	PCH-PIC + EIOINTC、IOCSR IPI	VirtIO PCI 块设备与网卡
VisionFive 2	OpenSBI + U-Boot，4 个 U74	PLIC、SBI timer/IPI	UART、SD/MMC、GMAC1
2K1000LA	U-Boot，2 个 LA264	LIOINTC、IOCSR IPI	UART、RTC、AHCI、GMAC0

表 8-1　MossOS 四个目标平台的主要硬件路径

8.2 设备树与硬件资源发现

8.2.1 设备树的传递与解析

物理平台由 OpenSBI 或 U-Boot 将设备树二进制数据（DTB）传给内核。早期启动代码先保留 DTB 物理地址，建立足以访问内核和 DTB 的临时映射，再进入通用运行时。`axhal::dtb` 向平台层和驱动层提供统一解析入口，同时将 DTB 本身及固件保留区排除在页分配器之外。

QEMU 目标的机器结构稳定，主要资源可由平台配置给出；物理板则需要同时考虑固件实际交付的拓扑。因此 MossOS 使用“板级约束 + DTB 资源发现”的方式：稳定不变的地址窗口由平台描述，可能随固件和板卡配置变化的信息从 DTB 获取。

资源发现与初始化流程如 图 8-2 所示。

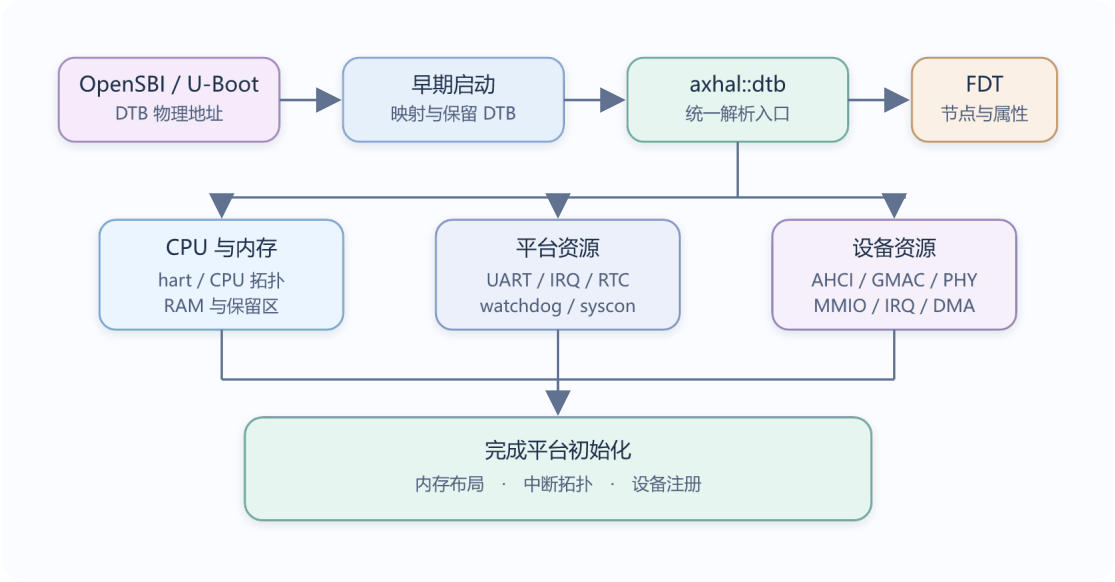


图 8-2　设备树传递与硬件资源发现

8.2.2 VisionFive 2 的硬件拓扑适配

VisionFive 2 平台描述给出 RAM、PLIC、UART、SD/MMC 和 GMAC1 的基本 MMIO 窗口，内核保留固件传入的 DTB，用于确认 CPU 与内存拓扑、固件保留区以及网卡 MAC 地址等信息。JH7110 的物理 hart 0 不提供普通 S-mode 运行环境，因此 MossOS 将逻辑 CPU 0–3 映射到物理 hart 1–4，并在中断和从核启动中始终使用这一映射。

8.2.3 2K1000LA 的动态资源发现

2K1000LA 的板级实现更多地依赖固件 DTB。平台层在早期、尚未启用动态内存分配时解析 CPU 硬件 ID、UART、LIOINTC、RTC、watchdog 以及 reboot/poweroff syscon；驱动则继续从 DTB 获取 AHCI 和 GMAC 的 MMIO、IRQ、PHY 与 DMA 参数。这样既避免在通用代码中写死样例板地址，也能根据 `status`、alias 和 phandle 选择实际启用的设备。

8.2.4 物理内存与 MMIO 布局

`MemIf` 统一描述可用 RAM、保留内存、MMIO 区间和物理/虚拟地址转换。VisionFive 2 的 4 GiB RAM 在早期由 Sv39 页表覆盖；2K1000LA 的 1 GiB 物理内存分布在低端和高端两个不连续区间，内核同时排除启动固件和 DTB 保留窗口。完成最终映射后，上层只通过 `axhal::mem` 访问普通内存和设备寄存器，不需要理解各板的地址窗口。

8.3 中断与多核事件机制

8.3.1 统一中断抽象与分发流程

CPU 进入 Trap 后，`axcpu` 先区分同步异常和中断，再将中断原因交给 `axhal::irq`。具体平台通过 `IrqIf` 完成处理函数注册、中断源使能、真实 IRQ 号获取以及硬件确认。定时器中断驱动调度时钟，IPI 用于远端唤醒和跨核 TLB 刷新，外部中断则按 IRQ 号进入 UART、存储或网卡驱动。

统一分发路径如 图 8-3 所示，四个平台仅在中断控制器和硬件确认方式上保留差异。

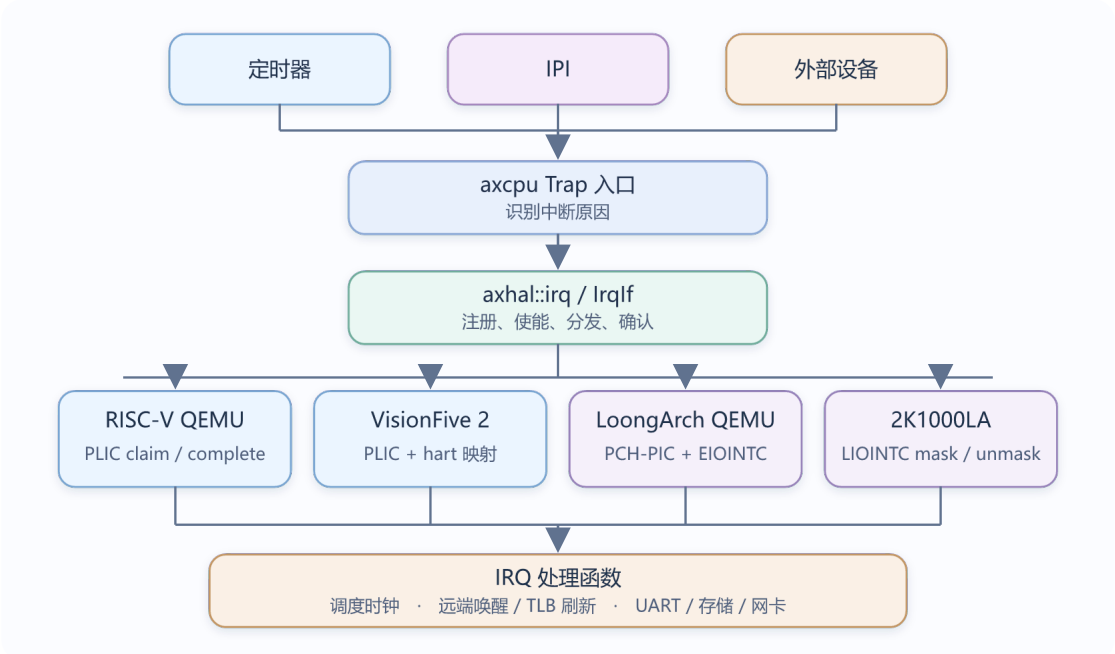


图 8-3　四平台的统一中断分发路径

8.3.2 RISC-V 平台的 PLIC 与 SBI

RISC-V 上层共享三类 S-mode 中断：STIP 用于定时器，SSIP 用于核间通知，SEIP 表示 PLIC 转发的外部设备中断。外部中断到达后，平台层先从 PLIC claim 寄存器获取真实 IRQ，调用处理函数后再 complete。定时器和 IPI 分别通过 SBI `set_timer` 与 `send_ipi` 提供。

RISC-V QEMU 上的 VirtIO MMIO 窗口对应 PLIC IRQ 1–8。VisionFive 2 也使用 PLIC，但需要将逻辑 CPU 转换为 JH7110 的物理 hart 和 S-mode PLIC context；其 UART、SD/MMC 和 GMAC1 都通过该路径产生设备中断。

8.3.3 LoongArch 平台的 EIOINTC、LIOINTC 与 IOCSR

LoongArch 的定时器由 CPU 稳频计数器和 `tcfg/ticlr` 寄存器控制，核间中断通过 IOCSR IPI 寄存器发送和确认。两个 LoongArch 平台的主要区别在外部中断路由：QEMU virt 使用 PCH-PIC 与 EIOINTC，2K1000LA 则使用由 DTB 描述的 LIOINTC。

2K1000LA 的 AHCI 和 GMAC 使用电平触发中断。LIOINTC 处理路径在调用设备处理函数前临时屏蔽当前源，待设备状态清除后再重新使能，避免未撤销的电平导致反复进入中断。

8.3.4 四个平台的中断机制对比

平台	定时器	**IPI**	外部中断
RISC-V QEMU	SBI timer / STIP	SBI IPI / SSIP	PLIC， VirtIO MMIO 和 UART
LoongArch QEMU	稳频计数器	IOCSR IPI	PCH-PIC + EIOINTC，VirtIO PCI 和 UART
VisionFive 2	SBI timer / STIP	SBI IPI / SSIP	PLIC，UART、SD/MMC 和 GMAC1
2K1000LA	稳频计数器	IOCSR IPI	LIOINTC，UART、AHCI 和 GMAC0

表 8-2　四个平台的中断路径对比

8.4 设备驱动

8.4.1 统一设备驱动框架

`axdriver` 通过 `DriverProbe` 统一设备探测，根据设备类型选择全局、MMIO 或 PCI 探测入口。探测成功的设备转换为 `AxDeviceEnum` 并注册到全局设备表，块设备向上实现 `BlockDriverOps`，网卡向上实现 `NetDriverOps`。因此 VFS 与 ext4 无需区分 VirtIO、SD/MMC 或 AHCI，网络栈也无需区分 VirtIO-net 和两种 GMAC。

8.4.2 QEMU 平台的 VirtIO 驱动

RISC-V QEMU 在固定 MMIO 窗口中枚举 VirtIO 设备，LoongArch QEMU 则通过 PCI ECAM 枚举 BDF，配置 BAR 后创建 VirtIO PCI 传输层。两条路径最终都进入同一 VirtIO 块设备或网卡实现。

VirtIO 通过 virtqueue 交换描述符链，`VirtIoHalImpl` 负责分配 DMA 页、转换物理地址和映射 MMIO。RISC-V VirtIO MMIO 可根据窗口地址得到 PLIC IRQ；LoongArch QEMU 中 VirtIO-blk 的 PCI legacy INTx 队列通知不够可靠，当前对块设备使用轮询完成，不影响平台定时器、IPI 和其他设备中断的正常工作。

8.4.3 VisionFive 2 的 SD/MMC 与 GMAC 驱动

VisionFive 2 的根文件系统位于 SD 卡。MossOS 在 `simple-sdmmc` 基础上将 JH7110 DWMMC 控制器接入块设备框架，补充了 PLIC 中断、等待队列、超时恢复、边界检查和 `flush()` 语义。当前决赛配置已启用控制器 IDMAC，使用对齐的描述符区和 bounce buffer 完成块数据传输。

网络方面，MossOS 为 JH7110 GMAC1 完成时钟与复位接管、RGMII 模式配置、MAC 地址获取、TX/RX 描述符环和 PLIC 中断接入。硬中断只记录设备事件并唤醒网络路径，报文和描述符回收在后续 poll 中完成，避免在中断上下文执行过多工作。UART 驱动也按 JH7110 的 4 字节寄存器步长完成适配，并使用接收中断和软件环形缓冲区支持交互式终端。

8.4.4 2K1000LA 的 AHCI 与 GMAC 驱动

2K1000LA 使用板载 SATA 磁盘作为根存储。MossOS 在 `simple-ahci` 基础上完成 AHCI 控制器探测、端口识别、中断完成和可睡眠等待，并保留有界轮询作为中断遗失时的恢复路径。驱动能够解析 MBR/GPT，按 `/dev/sdaN`、`PARTUUID` 或 `PARTLABEL` 选择 ext4 根分区，再将该分区封装为上层可直接使用的块设备。

GMAC0 驱动从 DTB 获取控制器、PHY、RGMII、MAC 地址和 DMA burst 参数，通过增强型描述符环完成收发，并将 DMA 事件接入 LIOINTC。除 AHCI 和 GMAC 外，平台还实现了 DTB 驱动的 UART、RTC、watchdog 以及 syscon 复位和关机路径，使内核能够完整接管板级运行环境。

8.4.5 DMA 与缓存一致性

DMA 正确性是物理设备驱动与 QEMU 路径的主要差异之一。VisionFive 2 的 SD/MMC IDMAC 将静态描述符和 bounce buffer 放在 `.dma_uncached` 段；GMAC1 在 CPU 与 DMA 交换缓冲区所有权时，通过 JH7110 L2 cache 操作和 `fence iorw, iorw` 维护一致性。2K1000LA 的 AHCI 和 GMAC 校验 DTB 中的 DMA mask，使用非缓存 DMW 访问设备数据，并通过 `dbar 0` 保证寄存器与描述符的可见顺序。

这些处理被封装在各驱动的 HAL 实现内，块设备框架、网络栈和用户程序不需要感知底层缓存是否一致。

8.4.6 驱动验证与能力边界

平台	存储路径	网络路径	当前验证边界
RISC-V QEMU	VirtIO-blk MMIO	VirtIO-net MMIO	已完成决赛用户态负载验证
LoongArch QEMU	VirtIO-blk PCI	VirtIO-net PCI	已完成决赛用户态负载验证
VisionFive 2	SD/MMC	JH7110 GMAC1	已完成四核、存储、双向 TCP 和用户程序实板验证
2K1000LA	AHCI SATA	DWMAC GMAC0	已完成双核、AHCI、ext4 和用户态实板验证；GMAC 数据面仍需补充完整压力验证

表 8-3　四个平台的设备驱动与验证边界

四个平台并不共享相同的启动流程、中断控制器和设备数据路径。MossOS 统一的是平台接口和上层设备语义：平台层将固件、内存和中断差异收敛到 `axplat`，驱动层将 VirtIO、SD/MMC、AHCI 和不同 GMAC 收敛为统一块设备与网络设备接口。因此，文件系统、网络栈、调度器和系统调用实现无需包含具体板级分支。

第 9 章 总结与展望

9.1 工作总结

MossOS 是面向 RISC-V64 和 LoongArch64 的 Linux ABI 兼容宏内核。系统在 ArceOS 和 StarryOS 基础上，形成了包含进程线程、任务调度、虚拟内存、文件系统、信号、同步和网络在内的内核主干，能够为 glibc 用户态程序提供基本的 Linux 运行环境。

决赛的 CAgent 与 BuildStorm 从不同角度验证了这套内核。CAgent 包含基础命令执行、时间处理、系统信息查询、网络状态查询以及文件和目录操作，侧重功能覆盖与 Linux ABI 兼容性。MossOS 在 RISC-V64 和 LoongArch64 上均通过了这些测试，说明进程执行、时钟、系统信息、网络和基本文件系统能够协同支撑常见用户态任务。

BuildStorm 要求内核在 8 核、8 GiB 环境中运行 Rust 工具链，从源码全量编译包含数百个 crate 的工程，侧重真实重负载下的正确性、并发性和性能。MossOS 在两种架构上均完成了工具链启动、最小工程构建和完整 BuildStorm 编译，验证了动态链接、进程与线程调度、虚拟内存、文件系统、多核同步及系统调用在长时间高并发场景下的综合能力。

在底层支持上，MossOS 进一步将同一套内核主体从双架构 QEMU 扩展到 VisionFive 2 和 2K1000LA 两个物理平台，完成了多核启动、中断、存储和物理网络等关键路径。这些结果表明，MossOS 不仅能够通过单项系统调用测试，也已具备运行综合用户程序、承载高并发负载并跨虚拟与真实硬件复用的基础。

9.2 未来工作

当前 MossOS 已经能够支撑决赛负载和多种物理设备，但与完整 Linux 语义仍有距离。进程管理还需完善非线程组组长发起的 `exec`、多线程组停止、namespace 隔离、capability 生命周期和 `ptrace` 数据访问，同时应将凭据校验统一接入各类信号接口。`seccomp` 可作为后续安全隔离和沙箱支持的重要方向。

内存和文件系统需要继续补全 `mremap` 的映射属性保留、真正的页面锁定、完整 `/proc/[pid]/maps` 信息和 ext4 xattr 持久化，并扩展 inotify/fanotify 的语义覆盖。网络方面将继续完善外部 IPv6、错误队列和部分 Socket 选项的实际行为。在现有同步 I/O 路径稳定后，还可以进一步研究 `io_uring` 等异步 I/O 接口。

性能方面，现有优化主要针对 BuildStorm 的读密集和高并发特征，后续需要在写密集、随机 I/O、长时间运行和内存压力场景下重新验证。调度器也可以在 RR 基础上引入更完整的优先级和负载平衡策略，继续细化文件描述符、进程树和缓存路径的锁粒度。

平台方面的下一步重点是建立 QEMU 与实板的自动化回归，对存储、物理网络、DMA 一致性、中断丢失和异常掉电进行长时间压力测试。同时，需继续完善 LoongArch QEMU 的 VirtIO 中断路径，并根据需求扩展 VisionFive 2 和 2K1000LA 的更多外设支持，使现有的四平台能力从“可运行”进一步走向可持续验证和长期稳定运行。